

LeRobot SO-101을 활용한
지능형 로봇 팔 제어
최종 보고서



팀명: Formula 1

202255576 윤주연 forsterix104@gmail.com

202155503 강태훈 kangth2103@gmail.com

202355713 송명근 smk00513@naver.com

지도교수: 김종덕 교수

제출일: September 13, 2026

부산대학교

목차

1	서론	1
1.1	지능형 로봇 시스템의 발전과 필요성	1
1.1.1	산업 현장에서의 로봇 조작 작업	1
1.1.2	지능형 자율 제어로의 패러다임 전환	1
1.2	로봇 조작의 기술적 어려움	2
1.2.1	환경적 불확실성과 객체 인식	2
1.2.2	고차원적 경로 계획 및 수직 적재 작업의 복잡성	2
1.3	모방학습 기반 로봇 제어	2
1.3.1	규칙 기반 제어에서 AI 기반 제어로의 전환	2
1.3.2	AI 기반 로봇 제어 방법과 모방학습의 선택	3
1.3.3	기존 연구에서의 의의와 한계	4
1.4	본 연구의 목적	5
2	연구 배경	6
2.1	모방 학습 (Imitation Learning)	6
2.1.1	행동 복제 (Behavioral Cloning)	6
2.1.2	역강화학습 (Inverse Reinforcement Learning)	7
2.1.3	로봇 제어 패러다임과 모방 학습의 적용	7
2.2	ACT (Action-Chunking Transformer)	7
2.2.1	ACT의 모델 구조 및 학습 과정	8
2.2.2	ACT의 실시간 추론 및 제어 과정	9
2.3	DAgger Correction (Data-Aggregation Correction)	9
3	연구 내용	11
3.1	로봇 시스템 환경 구축	11
3.1.1	SO-101 Leader-Follower 기반 Teleoperation 환경 구성	11
3.1.2	Jetson Orin Nano 기반 엣지 제어 환경 구축	13

3.1.3	상단/손목 카메라 연동 및 로봇 Calibration	13
3.1.4	Belly 카메라 도입 시도 및 하드웨어 제약 분석	14
3.1.5	학습 서버 및 개발 환경(도커 컨테이너) 구성	15
3.2	Teleoperation 기반 데이터 수집 및 데이터셋 구축	16
3.2.1	Pick & Stack 시연 데이터 수집 절차	16
3.2.2	외부 오픈소스 데이터셋 호환성 검토 및 자체 환경 동기화	16
3.2.3	작업 연속성을 고려한 시연 에피소드(Episode) 설계 및 수집 전략	17
3.2.4	블록 위치 조건별 데이터 수집	20
3.2.5	블록 탐색 및 선택 정책 결정 과정	23
3.2.6	시연 데이터 수집 매뉴얼	25
3.2.7	데이터 품질 관리 및 비정상 에피소드 검증·제거	27
3.2.8	LeRobot 데이터셋 구조 분석 및 병합·통합 관리	28
3.3	ACT 기반 모방학습 모델 구축	30
3.3.1	머신러닝 학습 종류 및 선정 과정	30
3.3.2	모방학습 모델 종류 및 선정 과정	32
3.3.3	Action Chunking Transformer(ACT) 기반 정책 구성	33
3.3.4	학습 파라미터 설정	34
3.3.5	학습 파라미터 변경에 따른 결과 변화 - TODO	36
3.3.6	Training → HF → Deployment	36
3.4	서버-클라이언트 기반 실시간 추론	37
3.4.1	서버-클라이언트 분산 추론 구조 설계	37
3.4.2	TCP Socket 기반 통신 프로토콜 및 Jetson Client 구현	37
3.4.3	서버 측 ACT 정책 추론 및 카메라 영상 전송 구현	38
3.5	실시간 추론 및 제어 성능 최적화	39
3.5.1	구간별 지연시간 측정	39
3.5.2	병목 구간 분석 및 개선	40
3.5.3	Action Chunk 및 Temporal Ensembling 기반 제어 방식 실험	40
3.6	ACT 모델 실패 원인 분석 및 DAgger	42
3.6.1	초기 모델의 수행 실패 유형 분석	42
3.6.2	CVAE 구조 관점에서의 Posterior Collapse 추정 및 하이퍼파라미터 실험	43
3.6.3	기존 행동 복제의 한계와 여러 대안의 검토, 그리고 DAgger의 선택	44
3.6.4	시스템 파이프라인에 DAgger가 적용되는 위치와 방법	45

3.6.5	Dagger 기반 실패 구간 보정 데이터 수집 및 반영 효과 검증	46
3.7	Task 조건화를 위한 통합 데이터셋 및 Environment State	47
3.7.1	단일 정책을 이용한 Task 1 · Task 2 수행	47
3.7.2	Environment State를 이용한 Task 조건화 설계	48
3.7.3	통합 ACT 모델 학습 및 Task별 추론	48
3.8	로봇 제어 안전 로직	49
3.8.1	정책 출력 직접 제어의 위험성 및 안전 로직 필요성	49
3.8.2	관절별 안전 범위 및 Step당 최대 변화량 제한 구현	49
3.8.3	LeRobot 프레임워크 내장 안전 기능과의 병행 적용	50
4	연구 결과 분석 및 평가	51
4.1	데이터셋 구축 결과	51
4.2	ACT 모델 학습 및 성능 평가	51
4.2.1	초기 ACT 모델 수행 결과	52
4.2.2	Dagger 적용에 따른 성능 변화	52
4.2.3	하이퍼파라미터 실험 결과	53
4.3	통합 ACT 모델의 Task 수행 평가	53
4.3.1	Task 1 수행 결과	53
4.3.2	Task 2 수행 결과	53
4.3.3	Environment State 기반 Task 조건화 평가	54
4.4	실시간 추론 시스템 성능 평가	54
4.4.1	로컬 추론과 서버 추론 비교	54
4.4.2	실시간 제어 지연시간 분석	54
5	결론 및 향후 연구 방향	56
5.1	연구 결론 요약	56
5.2	연구의 한계점	56
5.2.1	데이터셋 수집에서의 한계	56
5.2.2	학습 과정에서의 한계	57
5.3	향후 연구 방향	57
5.3.1	강화학습 기반 태스크 최적화	57
5.3.2	Sim-to-Real 도입	57
6	추진 체계 및 수행 결과	59

6.1	구성원별 수행 결과	59
6.2	개발 수행 일정	60

제1장 서론

1.1 지능형 로봇 시스템의 발전과 필요성

1.1.1 산업 현장에서의 로봇 조작 작업

로봇 매니퓰레이터는 제조 및 물류 환경에서 반복적인 물체 조작 작업을 자동화하기 위한 수단으로 활용되고 있다. 특히 물체를 집어 지정된 위치로 이동시키는 Pick-and-Place 작업은 로봇 팔을 활용한 대표적인 작업으로, 생산 공정의 자동화와 작업 효율 향상을 위해 사용되고 있다. 이러한 작업은 작업 대상의 위치와 작업 순서가 미리 정해진 환경에서는 비교적 안정적으로 수행할 수 있으며, 반복적인 작업을 자동화하는 데 적합하다.

그러나 실제 작업 환경에서는 물체가 항상 동일한 위치에 배치되거나 정해진 자세를 유지하는 것은 아니다. 작업 대상의 위치가 달라지거나 주변 환경에 변화가 발생할 경우, 사전에 정의된 동작만으로는 이러한 변화에 대응하기 어렵다. 이에 따라 로봇이 정해진 동작을 반복하는 것에서 나아가 작업 환경을 인식하고, 현재 상황에 적절한 행동을 수행할 수 있는 지능형 로봇 시스템의 필요성이 증가하고 있다.

1.1.2 지능형 자율 제어로의 패러다임 전환

기존의 로봇 시스템은 작업 환경과 대상이 일정하다는 가정하에 미리 정의된 궤적과 순서에 따라 로봇을 제어하는 방식이 주로 사용되었다. 이러한 방식은 정형화된 환경에서 높은 반복성과 안정성을 확보할 수 있지만, 작업 대상의 위치나 주변 환경이 변화하는 경우 새로운 조건에 대응하기 위해 제어 규칙과 동작을 추가로 설계해야 한다.

최근에는 카메라와 같은 비전 센서와 인공지능 기술을 로봇 제어에 활용하여 이러한 한계를 보완하고자 하는 연구가 이루어지고 있다. 로봇이 센서를 통해 주변 환경을 관찰하고, 인공지능을 이용하여 관측된 정보를 바탕으로 행동을 결정하는 방식으로 제어 시스템의 범위가 확장되고 있다. 이러한 변화는 로봇이 정해진 환경에서 반복 작업을 수행하는 자동화 시스템에서 벗어나, 변화하는 실제 환경에 대응할 수 있는 지능형 시스템으로 발전하는 과정으로 볼 수 있다.

1.2 로봇 조작의 기술적 어려움

1.2.1 환경적 불확실성과 객체 인식

실제 환경에서 로봇이 물체를 안정적으로 조작하기 위해서는 작업 대상의 위치와 상태를 정확하게 파악해야 한다. 특히 물체가 작업 공간 내에 무작위로 배치되는 경우 로봇은 카메라를 통해 물체의 위치를 파악하고, 인식된 정보를 실제 로봇의 움직임으로 연결해야 한다. 이 과정에서는 카메라와 로봇 사이의 좌표 관계를 고려해야 하며, 객체 인식이나 좌표 변환 과정에서 발생한 오차가 이후의 파지 및 이동 과정에 영향을 줄 수 있다.

또한 실제 작업에서는 물체가 서로 가까이 배치되거나 일부가 다른 물체에 의해 가려질 수 있으며, 카메라의 위치와 촬영 환경에 따라 관측 결과가 달라질 수 있다. 조명이나 영상 환경의 변화 역시 객체 인식 성능에 영향을 줄 수 있다. 따라서 로봇 조작 시스템에서는 단순히 물체를 탐지하는 것뿐만 아니라, 다양한 환경에서 얻어진 인식 결과를 로봇의 제어에 안정적으로 활용할 수 있는 방법이 필요하다.

1.2.2 고차원적 경로 계획 및 수직 적재 작업의 복잡성

물체의 위치를 인식한 이후에도 안정적인 조작을 위해서는 적절한 파지 동작과 이동 경로를 생성해야 한다. 로봇 팔이 블록을 정확하게 파지하지 못하거나 이동 과정에서 위치 오차가 발생하면 물체가 이탈하거나 목표 영역에 정확하게 배치되지 않을 수 있다. 또한 여러 개의 블록을 순차적으로 적재하는 경우에는 이미 배치된 블록의 위치와 높이를 함께 고려해야 하므로 단순한 이동 작업보다 높은 제어 정밀도가 요구된다.

특히 Pick-and-Stack 작업에서는 각 단계에서 발생한 작은 위치 오차가 이후의 적재 과정에 누적될 수 있다. 블록의 중심이 정확하게 정렬되지 않은 상태에서 다음 블록을 적재할 경우 전체 적재물의 안정성이 저하될 수 있으며, 로봇의 이동 과정에서도 기존에 적재된 블록과의 충돌을 고려해야 한다. 따라서 이러한 작업을 안정적으로 수행하기 위해서는 개별 동작뿐만 아니라 파지, 이동, 배치 및 다음 작업으로의 전환을 포함한 전체 작업 과정을 함께 고려할 필요가 있다.

1.3 모방학습 기반 로봇 제어

1.3.1 규칙 기반 제어에서 AI 기반 제어로의 전환

로봇의 동작을 제어하는 전통적인 방법은 사람이 작업 과정을 분석하고 각 상황에 필요한 조건과 동작을 직접 정의하는 규칙 기반(Rule-based) 방식으로 구현할 수 있다. 예를 들어

특정 위치에 물체가 존재하면 정해진 경로를 따라 로봇 팔을 이동시키고, 지정된 조건을 만족하면 그리퍼를 작동시키는 방식으로 작업 절차를 구성할 수 있다. 이러한 방식은 작업 과정과 환경이 명확하게 정의되어 있는 경우 구현과 검증이 비교적 용이하다는 장점이 있다.

그러나 작업 대상의 위치가 달라지거나 예상하지 못한 상황이 발생하는 경우에는 각각의 상황에 대응하기 위한 규칙을 추가해야 한다. 특히 물체의 파지와 적재처럼 여러 관절의 움직임이 연속적으로 이루어지는 작업에서는 각 단계의 동작과 예외 상황을 사람이 직접 정의하는 데 한계가 있다. 이에 따라 최근에는 사람이 모든 동작 규칙을 직접 작성하는 대신, 센서로부터 얻은 관측 정보와 로봇의 행동 사이의 관계를 인공지능 모델이 학습하도록 하는 AI 기반 로봇 제어 방식이 활용되고 있다.

이와 같은 접근은 인공지능이 디지털 환경의 데이터를 처리하는 것에 그치지 않고, 카메라와 센서 등을 통해 물리적 환경을 인식하고 실제 로봇의 행동으로 연결한다는 점에서 Physical AI의 개념과 연결된다. Physical AI는 인공지능을 물리적 세계에 적용하여 시스템이 주변 환경을 인식하고 이해하며, 상황에 따라 실제 행동을 수행하도록 하는 접근을 의미한다. 로봇 분야에서는 이러한 특성을 바탕으로 환경 인식과 판단, 행동 생성을 하나의 End-to-End 시스템으로 연결하는 방향으로 연구가 확대되고 있다.

1.3.2 AI 기반 로봇 제어 방법과 모방학습의 선택

AI 기반 로봇 제어에서는 로봇의 관측 정보로부터 적절한 행동을 생성하기 위해 다양한 학습 방법을 적용할 수 있다. 본 연구에서는 이 가운데 사람이 직접 수행한 로봇 조작 데이터를 활용하는 모방학습(Imitation Learning)에 주목하였다. 모방학습은 전문가 또는 사용자가 수행한 시연을 학습 데이터로 활용하여, 주어진 관측에 대해 해당 작업에 적합한 행동을 예측하도록 정책을 학습하는 방법이다.

로봇 조작 작업은 사람이 직접 수행하기에는 비교적 직관적이지만, 이를 수많은 규칙과 좌표로 표현하는 것은 복잡할 수 있다. 반면 사람이 직접 로봇을 조작하여 작업을 수행하고 그 과정에서 발생하는 영상 정보와 로봇 상태, 행동 데이터를 수집하면 실제 작업에 필요한 동작을 데이터의 형태로 표현할 수 있다. 따라서 Teleoperation을 이용한 시연 데이터 수집은 실제 로봇 조작 작업에 필요한 행동을 학습 데이터로 변환하는 적절한 방법이 될 수 있다.

특히 본 연구의 대상인 Pick-and-Place 및 Pick-and-Stack 작업은 사람이 직접 수행할 수 있는 조작 작업이며, 작업의 성공 여부를 판단할 수 있는 명확한 목표가 존재한다. 이에 따라 사람이 블록을 집고 이동시키는 과정을 직접 시연하고, 이를 반복적으로 수집하여 로봇의 행동 정책 학습에 활용하는 모방학습 방식을 적용하였다. 또한 LeRobot은 Leader Arm 등을 이용한 Teleoperation을 통해 로봇의 동작 궤적을 기록하고, 수집한 데이터를 기반으로 정책을 학습하여 실제 로봇에 배포하는 작업 흐름을 지원한다.

1.3.3 기존 연구에서의 의의와 한계

최근 오픈소스 로봇 플랫폼과 모방학습 프레임워크의 발전에 따라 다양한 로봇 조작 데이터셋과 사전 학습 모델이 공개되고 있다. 대표적으로 LeRobot 프레임워크 기반의 SO-100 로봇 암을 활용한 Pick-and-Place 시연 데이터셋인 `lerobot/svla_so100_pickplace`를 들 수 있다^{1,2}. 해당 데이터셋은 총 50개의 에피소드(약 19,631 프레임)로 구성되어 있으며, 이를 바탕으로 ACT(Action Chunking with Transformers) 아키텍처 및 SmolVLA와 같은 소형 비전-언어-행동(Vision-Language-Action, VLA) 모델의 학습과 평가가 활발히 진행되고 있다. 허깅페이스(Hugging Face) 커뮤니티에는 이 데이터를 활용해 파인튜닝된 어댑터 모델(예: `raja-kumar/smolvla_so100_pickplace_v2`)이 공유되는 등 유의미한 연구 생태계가 형성되어 있다.

이 데이터셋을 활용해 ACT 모델을 학습한 기존 연구는 50개의 고품질 시연 데이터만으로 기본 조건에서 약 70 ~ 80% 수준의 작업 성공률을 달성할 수 있음을 보여주었다. 이는 소규모 데이터셋으로도 모방학습 기반의 로봇 제어가 가능함을 입증한다는 점에서 큰 의의를 가진다.

그러나 이러한 성능은 조작 대상 객체의 시작 위치가 고정되어 있고, 카메라의 시점과 조명 조건이 동일하게 유지되는 매우 제한적인 환경을 전제로 한 결과라는 명확한 한계를 지닌다. 물체의 위치가 조금만 바뀌거나 환경적 변화가 개입될 경우, 50개의 에피소드만으로는 모델이 일반화를 수행하기 어려워 작업 실패율이 급격히 증가한다. 관련 로봇 커뮤니티의 공통된 평가에 따르면, 약간의 환경 변화 조건에서도 80% 이상의 안정적인 조작 성능을 확보하기 위해서는 최소 100개 이상의 다각화된 에피소드 데이터가 필수적인 것으로 분석된다.

더욱이 기존 `svla_so100_pickplace` 데이터셋 기반의 연구 환경과 본 연구에서 해결하고자 하는 과제 간에는 다음과 같은 뚜렷한 난이도 및 환경적 차이가 존재한다.

- **시각적 노이즈의 차이:** 기존 연구는 단일한 검은색 배경에서 수행되어 비전 인식의 난이도가 낮으나, 본 연구는 검은색과 흰색이 교차하는 격자무늬 배경을 사용하여 비전 모델의 강건한 시각적 특징 추출 능력이 요구된다.
- **태스크 난이도의 차이:** 단일 블록을 단순한 통 안에 넣는 기존 작업과 달리, 본 연구는 다섯 개의 블록을 연속적으로 파지하여 특정 영역 안에 배치하거나 수직으로 적재해야 하는 고도화된 정밀도와 작업 연속성을 요구한다.
- **태스크 수의 차이:** 단일 목적의 태스크만을 수행하는 기존 연구의 범위를 넘어, 본 연구는 두 가지 서로 다른 태스크를 단일 모델이 구분하고 동시에 성공적으로 수행할 수

¹https://huggingface.co/datasets/lerobot/svla_so100_pickplace

²<https://robocloud-dashboard.vercel.app/learn/blog/lerobot-act-pick-place-demonstrations>

있어야 한다.

결론적으로, 기존 연구가 지닌 데이터 편향 및 일반화의 한계점과 본 과제의 높은 복잡도를 고려할 때, 단순한 시연 데이터 수집만으로는 목표 성능을 달성하기 어렵다. 따라서 위와 같은 선행 연구와의 차이점 분석은, 본 연구에서 ACT 모델이 현재의 복잡한 문제를 성공적으로 수행하기 위해 어떤 전략으로 데이터셋을 구성하고 양을 늘려가야 하는지를 결정하는 핵심 마일스톤으로 작용할 것이다.

1.4 본 연구의 목적

본 연구의 목적은 LeRobot SO-101을 기반으로 작업 환경을 인식하고 물체를 자율적으로 조작할 수 있는 지능형 로봇 제어 시스템을 구축하는 것이다. 이를 위해 테이블 위에 무작위로 배치된 블록을 대상으로 카메라를 이용하여 작업 환경을 인식하고, 로봇 매니퓰레이터가 블록을 집어 지정된 영역으로 이동시키거나 기존 블록 위에 순차적으로 적재할 수 있도록 시스템을 구성하였다.

본 연구에서는 이러한 작업을 수행하기 위해 Leader-Follower 기반의 Teleoperation 환경을 구축하고, 사람이 직접 수행한 로봇 조작 데이터를 수집하여 모방학습에 활용하였다. 영상 정보와 로봇 관절 데이터를 포함하는 시연 데이터를 구축한 후, 이를 기반으로 로봇의 작업 수행에 필요한 행동 정책을 학습하고 실제 로봇에 적용하는 것을 목표로 하였다.

최종적으로 본 연구에서는 카메라를 통한 작업 환경 인식, 로봇 조작 데이터 수집, 모방 학습 기반 행동 학습 및 실제 로봇 제어를 연계함으로써, 무작위로 배치된 블록에 대해 Pick-and-Place 및 Pick-and-Stack 작업을 성공적으로 수행할 수 있는 지능형 로봇 제어 시스템을 구현하고자 하였다.

제2장 연구 배경

2.1 모방 학습 (Imitation Learning)

모방 학습(Imitation Learning, IL)은 에이전트가 명시적인 보상 함수 설계 없이, 인간 전문가의 시연 데이터를 바탕으로 최적의 행동 정책을 학습하는 기법이다. 전통적인 강화학습이 복잡한 로봇 조작 환경에서 보상 함수를 정의하기 어렵고 탐색 과정에서 많은 시간이 소요되는 반면, 모방 학습은 양질의 데이터셋을 통해 로봇이 전문가의 직관적인 제어 방식을 효과적으로 습득할 수 있게 한다. 모방 학습의 대표적인 접근법으로는 행동 복제와 역강화학습이 있다.

2.1.1 행동 복제 (Behavioral Cloning)

행동 복제는 모방 학습의 가장 직관적인 방법론으로, 전문가의 상태-행동(State-Action) 쌍 데이터를 지도 학습 방식으로 학습한다. 즉, 환경의 상태 s_t 가 주어졌을 때 전문가의 행동을 모사하도록 정책을 최적화한다. 모델 구현이 비교적 단순하고 빠른 학습이 가능하다는 장점이 있어 최근 딥러닝 및 비전 기반 End-to-End 로봇 제어 분야에서 널리 활용되고 있다.

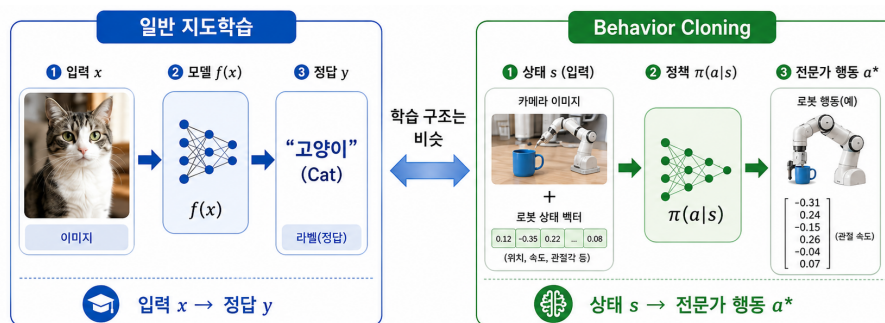


그림 2.1: 일반적인 지도 학습과 행동 복제의 차이

그림 2.1에서 볼 수 있듯이, 일반적인 지도 학습과 행동 복제는 데이터의 독립성과 예측 결과의 순차적 종속성 측면에서 결정적인 차이를 가진다. 일반 지도 학습은 각 데이터 샘플이 독립적이고 동일한 분포를 따른다고 가정하므로, 특정 시점의 예측 오차가 다음 입력 데이터에 영향을 미치지 않는다. 반면, 로봇 제어에서의 행동 복제는 에이전트가 예측한 현재 행동이

환경과 상호작용하여 다음 상태를 직접적으로 결정하는 마르코프 결정 과정(MDP)의 특성을 띤다.

이러한 순차적 의사결정 구조로 인해, 행동 복제는 전문가가 시연한 궤적 분포 내에서만 안정적인 성능을 보인다는 치명적인 한계를 지닌다. 모델이 추론 시점에 기존 분포를 벗어난 낮은 상태에 직면하거나 작은 오차를 발생시킬 경우, 이 오차로 인해 다음 입력 상태는 전문가의 데이터 분포에서 더욱 멀어지게 된다. 결과적으로 예측 오차가 시간이 지남에 따라 눈덩이처럼 증폭되는 누적 오차 및 공변량 이동 문제가 발생하여 연쇄적인 로봇의 오작동으로 이어진다.

2.1.2 역강화학습 (Inverse Reinforcement Learning)

역강화학습은 전문가의 시연 데이터로부터 달성하고자 하는 내재적인 보상 함수를 역으로 추정하고, 이를 바탕으로 강화학습을 수행하여 정책을 도출하는 방법이다. 보상 함수를 직접 알아냄으로써 환경 변화에 대해 행동 복제보다 더 강건한 일반화 성능을 기대할 수 있다. 하지만 모델의 연산 복잡도가 매우 높고 실시간 로봇 매니플레이션 환경에 적용하기에는 컴퓨팅 자원과 샘플 효율성 측면에서 제약이 따른다.

2.1.3 로봇 제어 패러다임과 모방 학습의 적용

최근 지능형 로봇 시스템은 복잡한 역운동학 기반의 전통적 제어 대신, 다중 카메라 영상과 같은 고차원 관측 데이터를 직접 입력받아 제어 명령을 출력하는 방식으로 발전하고 있다. 본 연구에서는 로봇 조작의 부드러움과 정확성을 극대화하기 위해 행동 복제 기반의 아키텍처를 채택하되, 고질적인 누적 오차 문제를 극복하기 위해 다중 궤적 예측과 데이터 집계 보정 기법을 함께 적용하여 시스템의 안정성을 확보하고자 한다.

2.2 ACT (Action-Chunking Transformer)

ACT(Action Chunking with Transformers)는 최근 로봇 조작 및 모방학습 분야에서 뛰어난 성능을 보이는 트랜스포머 기반의 정책 모델이다. 기존의 모방학습 모델들이 현재의 관측을 바탕으로 단일 스텝의 행동만을 예측했던 것과 달리, ACT는 한 번의 추론으로 일정 길이의 연속된 행동 시퀀스, 즉 행동 청크(Action Chunk)를 예측한다. 이를 통해 시계열적으로 발생하는 누적 오차를 줄이고, 로봇의 움직임이 멈추거나 떨리는 현상을 완화하여 작업의 연속성을 보장한다.

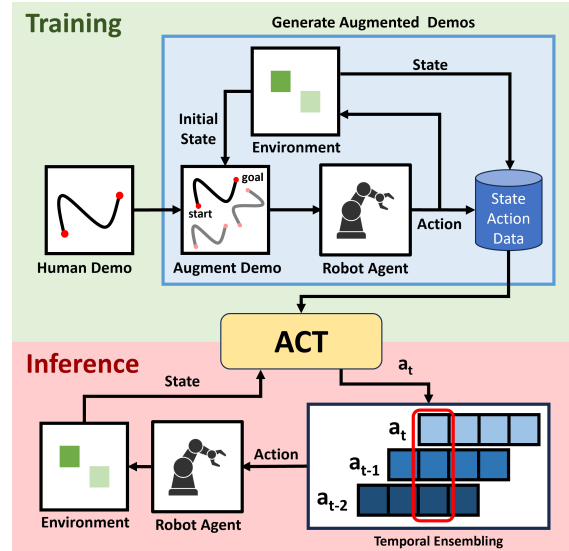


그림 2.2: ACT 모델의 전반적인 구조: 학습 과정과 실시간 추론 과정

2.2.1 ACT의 모델 구조 및 학습 과정

ACT의 학습은 CVAE(Conditional Variational Autoencoder) 구조를 기반으로 이루어진다. 로봇 제어에서 발생하는 다형성(Multimodality, 동일한 목표를 수행하기 위한 다양한 궤적 존재)을 모델링하기 위해 잠재 변수 z 를 도입한다. 학습 과정은 구체적으로 다음과 같이 진행된다.

1. **CVAE 인코더 (Encoder):** 학습 시에는 전문가의 시연 데이터로부터 현재 시점의 관측값 (다중 카메라 영상, 로봇의 현재 관절 위치 등)와 미래의 행동 시퀀스가 함께 인코더에 입력된다. 인코더는 이 정보를 바탕으로 잠재 변수 z 의 분포를 추정한다.
2. **CVAE 디코더 (Decoder/Policy):** 트랜스포머 기반으로 구성된 디코더는 인코더에서 샘플링된 잠재 변수 z 와 현재 관측값을 조건으로 입력받는다. 디코더는 입력된 시연 데이터의 행동 시퀀스를 모사하여 예측된 행동 청크를 출력한다.
3. **손실 함수 (Loss Function):** ACT의 최적화를 위한 손실 함수는 두 가지 항목의 합으로 구성된다. 첫째는 예측된 행동 청크와 실제 전문가의 행동 청크 간의 차이를 최소화하는 L1 혹은 L2 복원 손실이다. 둘째는 인코더가 추정한 z 의 분포가 사전 분포(Prior distribution, 일반적으로 표준 정규 분포와 유사해지도록 유도하는 KL 발산(Kullback-Leibler Divergence) 손실이다.

이러한 CVAE 기반의 학습을 통해 ACT 모델은 단순한 행동의 암기를 넘어, 잠재 공간(Latent Space) 상에서 로봇의 다양한 행동 궤적을 일반화하여 학습하게 된다.

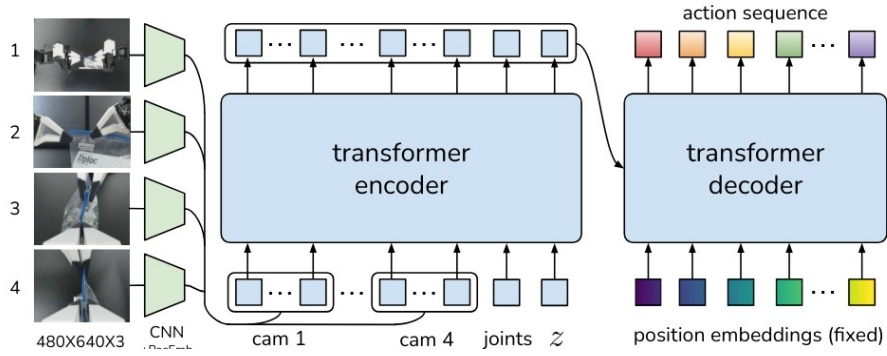


그림 2.3: ACT 모델의 학습 구조

2.2.2 ACT의 실시간 추론 및 제어 과정

실제 환경에서 로봇을 제어하는 추론 단계에서는 미래의 행동 시퀀스에 대한 정보가 존재하지 않으므로, 학습 과정과는 다른 파이프라인으로 동작한다.

1. **잠재 변수 설정 (Latent Variable Sampling):** 미래 행동을 알 수 없기 때문에 CVAE 인코더를 사용할 수 없다. 대신, 학습 과정에서 KL 발산 손실을 통해 정규화해둔 사전 분포로부터 z 값을 샘플링하거나, 결정론적 성능을 위해 z 를 단순히 0의 벡터로 설정하여 디코더에 입력한다.
2. **행동 청크 생성 (Action Chunk Generation):** 트랜스포머 디코더는 설정된 잠재 변수 z 와 카메라 센서 및 로봇 관절 인코더로부터 획득한 현재 관측값만을 이용하여 미래 스텝의 행동 청크를 한 번에 예측한다.
3. **시간적 앙상블 (Temporal Ensembling):** 매 제어 주기마다 새로운 행동 청크가 예측되므로, 시계열 상에서 이전 스텝에 예측한 미래의 행동들과 현재 스텝에서 예측한 행동들이 겹치는 구간이 발생한다. ACT는 이 중첩된 예측값들을 지수 가중 평균 등을 활용하여 앙상블하여 최종 목표 관절 위치를 결정한다.

이러한 Temporal Ensembling 기법은 특정 시점의 추론에서 일시적인 노이즈나 아웃라이어가 발생하더라도, 다른 시점의 예측값들이 이를 보완해주어 부드럽고 안정적인 로봇 제어를 가능하게 한다.

2.3 DAgger Correction (Data-Aggregation Correction)

전통적인 모방학습 기법인 행동 복제는 전문가의 시연 궤적을 지도학습 방식으로 직접 모사하는 직관적인 방법이다. 그러나 학습된 정책이 실제 환경에서 제어를 수행할 때, 미세한

예측 오차가 궤적 전체에 걸쳐 누적되면서 에이전트가 전문가 데이터셋에 존재하지 않는 낮은 상태분포로 진입하게 되는 누적 오차 문제가 빈번하게 발생한다.

이러한 상태에서 에이전트는 올바른 대처 방법을 알지 못해 결국 작업 수행에 실패하게 되며, 이는 수직 적재와 같은 고도의 정밀도가 요구되는 로봇 조작 작업에서 치명적인 한계로 작용한다. 이를 극복하기 위해 제안된 대표적인 알고리즘이 DAgger(Dataset Aggregation)이다.

DAgger는 에이전트의 현재 정책을 기반으로 환경과 상호작용하며 새로운 궤적을 수집하고, 이 새로운 상태 분포에 대해 전문가가 올바른 행동을 라벨링하여 기존 데이터셋에 지속적으로 통합하는 반복적 메타 알고리즘이다. 특히 로봇 원격 조작 환경에서의 DAgger Correction 수행 절차는 다음과 같이 요약할 수 있다.

1. **초기 정책 학습:** 초기 전문가 시연 데이터셋을 이용하여 첫 번째 정책을 학습한다.
2. **정책 실행 및 궤적 수집:** 학습된 정책을 제어 루프에 배포하여 로봇을 구동시키고, 에이전트가 방문하는 새로운 상태 데이터를 수집한다.
3. **전문가 개입 및 교정:** 로봇이 최적 궤적을 이탈하거나 실패가 예상되는 비정상적인 상태에 직면했을 때, 전문가가 즉각적으로 개입하여 원래의 궤적으로 복귀하기 위한 올바른 제어 명령을 제공한다.
4. **데이터 병합:** 에이전트가 경험한 상태와 전문가가 교정한 행동 쌍으로 구성된 새로운 데이터셋을 기존 데이터셋에 병합한다.
5. **정책 재학습:** 확장된 데이터셋을 사용하여 다음 정책을 학습하며, 로봇이 원하는 성능에 도달할 때까지 위 과정을 반복한다.

로봇 조작 분야에서 DAgger 기반의 교정 데이터는 단순한 데이터 볼륨의 증가를 의미하는 것이 아니다. 에이전트에게 "실수했을 때 원래의 궤도로 복구하는 방법"을 명시적으로 학습시킴으로써, 제한된 초기 시연 데이터만으로는 학습하기 어려운 강건성과 일반화 성능을 모델에 부여한다는 점에서 중요한 의미를 지닌다.

제3장 연구 내용

3.1 로봇 시스템 환경 구축

3.1.1 SO-101 Leader-Follower 기반 Teleoperation 환경 구성

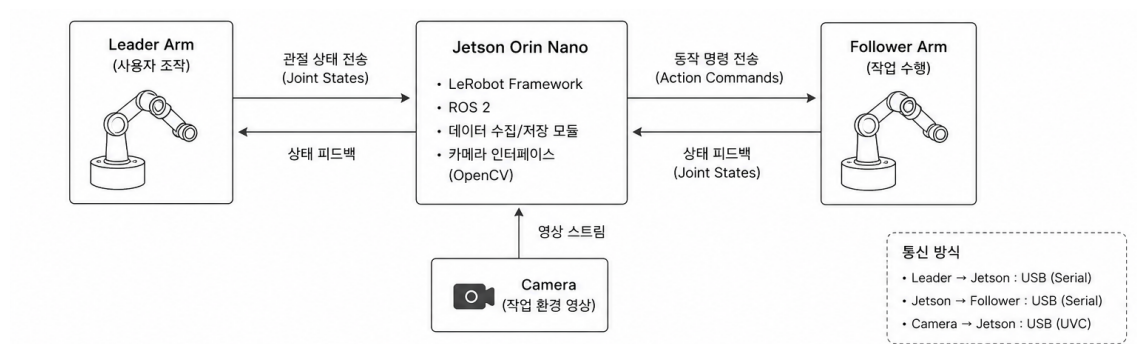


그림 3.1: Jetson Orin Nano 중심 시스템 및 통신 흐름

본 연구에서는 LeRobot 프레임워크에서 제공하는 SO-101 로봇팔을 사용하였다. [1] SO-101은 리더(Leader)-팔로워(Follower) 구조로 구성되며, 사용자가 리더 암을 직접 조작하면 팔로워 암이 동일한 관절 각도로 이를 추종하는 방식으로 동작한다.[2] 이러한 구조는 별도의 원격 제어 인터페이스 없이도 직관적인 시연 데이터 수집을 가능하게 하며, 본 연구에서 사용한 모방학습(imitation learning) 기반 정책 학습의 핵심 데이터 수집 수단으로 활용하였다.



그림 3.2: Leader Arm



그림 3.3: Follower Arm

로봇의 각 관절은 6자유도(shoulder_pan, shoulder_lift, elbow_flex, wrist_flex, wrist_roll, gripper)로 구성되며, 리더 암의 조작값이 팔로워 암에 실시간으로 전달되도록 lerobot-record 명령을 통해 Teleoperation 환경을 구성하였다. 이를 통해 사용자가 리더 암으로 수행한 블록 집기 및 적재 동작을 팔로워 암이 추종하도록 하고, 해당 관절 상태와 카메라 관측 데이터를 시연 데이터로 기록하였다.

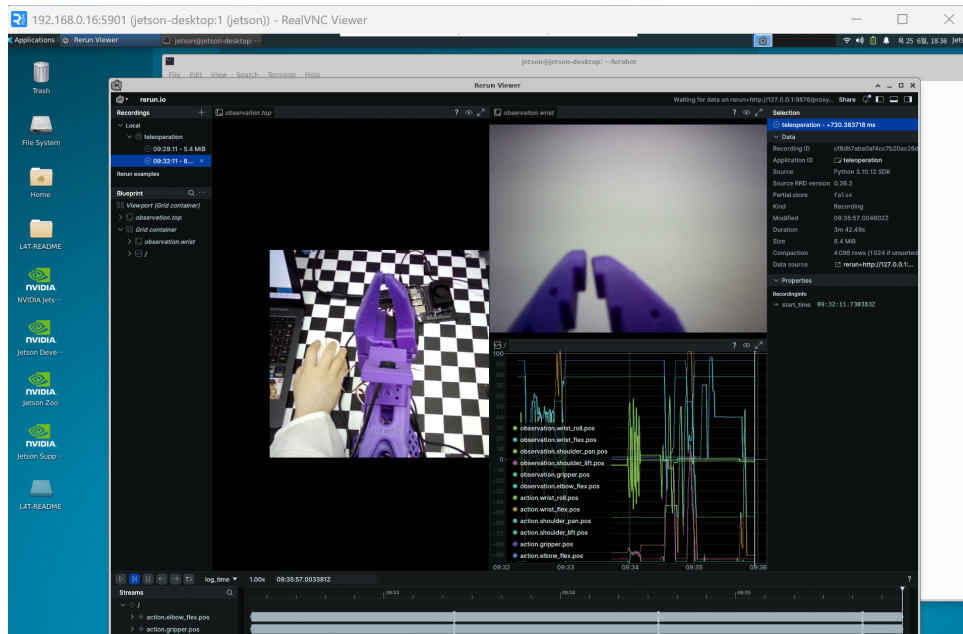


그림 3.4: Teleoperation 및 카메라 연동 화면

3.1.2 Jetson Orin Nano 기반 엣지 제어 환경 구축

로봇의 실시간 제어 및 데이터 수집은 배부된 Jetson Orin Nano(JetPack6.2.2, ARM64 아키텍처) 상에서 수행하였다. [3] Jetson 환경은 일반적인 x86 기반 학습 서버와 하드웨어 아키텍처 및 GPU 환경이 상이하므로, 로봇 제어 및 추론에 필요한 라이브러리를 Jetson 환경에 맞게 구성하는 과정이 필요하였다. [4]

특히 초기 환경에서는 PyTorch의 GPU 연산 환경이 Jetson Orin Nano의 CUDA 및 GPU 아키텍처와 호환되지 않아 추론 과정에서 오류가 발생하였다. 이에 따라 Jetson 환경에 맞는 PyTorch 배포 버전을 적용하고 CUDA 기반 GPU 연산 환경을 재구성하여 로봇 추론 및 카메라 데이터 처리가 가능하도록 환경을 구축하였다. 로봇의 실제 제어는 Jetson Orin Nano에서 수행하고, 모델 학습은 별도의 GPU 서버에서 수행하는 분산 환경을 구성함으로써 학습 환경과 실제 로봇 제어 환경을 분리하였다.

3.1.3 상단/손목 카메라 연동 및 로봇 Calibration

팔로워 암에는 작업 공간 전체를 관측하는 상단(top-view) 카메라와 그리퍼 근접 시점을 관측하는 손목(wrist-view) 카메라 총 2대를 연동하였다. 두 카메라는 640x480 해상도, 30FPS를 기준으로 구성하였으며, 각각 서로 다른 관측 범위를 담당하도록 구성하였다.

상단 카메라는 작업 공간 내 블록과 지정된 적재 영역의 전체적인 위치를 파악하는데 활용하였으며, 손목 카메라는 그리퍼가 블록에 접근하고 집는 과정에서 근접한 시각 정보를 제공하도록 구성하였다. 이러한 두 시점의 영상을 동시에 활용함으로써 단일 카메라만 사용하는 경우보다 작업 공간의 전반적인 상태와 그리퍼 주변의 세부적인 상태를 함께 관측할 수 있도록 하였다.



그림 3.5: Pick & Stack 실험 환경

또한 로봇의 관절 캘리브레이션(Calibration)은 각 관절의 최소 및 최대 가동 범위를 기준으로 수행하였다. 캘리브레이션 값은 로봇의 실제 관절 상태와 제어 명령 간의 관계를 결정하므로, 데이터 수집 및 추론 과정에서 동일한 캘리브레이션 기준을 유지하는 것이 중요하다.

실험 과정에서는 캘리브레이션 값이 변경되거나 기존 값이 유실될 경우 학습된 정책이 기대하는 로봇의 관절 좌표와 실제 로봇의 동작 사이에 차이가 발생할 수 있음을 확인하였다. 이에 따라 캘리브레이션 값의 유실에 대비하여 백업본을 별도로 관리하고, 데이터 수집 및 추론 시 동일한 캘리브레이션 기준을 사용하도록 구성하였다.

3.1.4 Belly 카메라 도입 시도 및 하드웨어 제약 분석

프로젝트 중간보고서 단계에서 작업 영역의 시야각 확장과 인지 성능 향상을 위해 기존 상단 카메라(Top Camera)와 손목 카메라(Wrist Camera) 외에 벨리 카메라(Belly Camera)를 추가로 도입할 계획을 수립한 바 있다. 그러나 실제 환경에서 이를 적용하기 위한 사전 테스트 과정에서 다음과 같은 물리적 한계와 하드웨어 제약이 발생하여 최종적으로 해당 카메라의 도입을 취소하게 되었다.

첫째, 본 프로젝트의 핵심 연산 장치인 Jetson Orin Nano는 총 4개의 USB-A 포트를 제공한다. 시스템 운용을 위해서는 리더암(Leader Arm), 팔로워암(Follower Arm)의 제어 통신과 두 대의 기존 카메라(Top, Wrist) 연결이 필수적이며, 이로 인해 이미 4개의 포트가 모두 점유되어 여유 포트가 없는 상황이었다.

둘째, 포트 부족 문제를 해결하기 위해 USB 허브를 활용한 포트 확장을 시도하였다. 특히

Jetson Orin Nano의 전원 공급 한계를 고려하여 안정적인 전력 공급이 가능한 외부 유전원 USB 허브를 채택하여 하드웨어적 대비를 갖추고자 하였다.

셋째, Belly 카메라가 도입되기 전 시스템 안정성을 검증하기 위해 기존의 두 카메라를 유전원 허브에 연결하여 사전 테스트를 진행하였다. 그러나 다수의 고해상도 비전 데이터를 단일 USB 버스(Bus) 채널로 통합하여 전송하는 과정에서 데이터 병목(Bottleneck) 현상 및 대역폭(Bandwidth) 포화 문제가 발생하였다. 이로 인해 통신 지연이 유발되어 로봇 팔의 제어 명령이 정상적으로 송수신되지 않고 오작동을 일으키는 치명적인 문제가 확인되었다.

결론적으로, 하드웨어 데이터 전송 대역폭의 물리적 한계로 인해 추가적인 비전 센서의 도입은 전체 로봇 제어 시스템의 실시간성과 안정성을 심각하게 훼손할 것이라는 기술적 판단을 내렸다. 이에 따라 Belly 카메라의 추가 도입을 포기하는 대신, 기존에 안정성이 확보된 2개의 카메라(Top, Wrist) 구성으로 복귀하여 데이터의 활용도를 극대화하고 알고리즘 및 경로 계획을 최적화하는 방향으로 연구의 초점을 수정하여 최종 과제를 진행하였다.

3.1.5 학습 서버 및 개발 환경(도커 컨테이너) 구성

정책 학습은 별도의 GPU 서버(NVIDIA RTX 5090, 32GB VRAM) 상에서 수행하였다. 학습 서버는 도커 컨테이너 형태로 제공되었으며, 컨테이너 내부에 Conda 기반 가상환경을 구성하여 Python, PyTorch, LeRobot 등 정책 학습에 필요한 라이브러리를 관리하였다.

Jetson Orin Nano는 ARM64 기반의 엣지 컴퓨팅 환경인 반면 학습 서버는 GPU 서버 환경으로 구성되어 있어 두 시스템의 하드웨어 및 소프트웨어 환경이 서로 상이하였다. 따라서 학습 서버에서 학습한 정책을 실제 Jetson 환경으로 배포할 때에는 정책의 설정 파일(config)과 라이브러리 버전 등의 호환성을 확인하는 과정을 수행하였다.

본 연구에서는 초기에는 Jetson Orin Nano에서 학습된 ACT 정책을 직접 실행하는 로컬 추론 방식으로 시스템을 구성하였다. 이후 실제 로봇 제어 과정에서 추론 연산과 카메라 영상 처리 및 로봇 제어를 하나의 엣지 장치에서 수행하는 구조의 실시간 처리 성능을 비교·검토하였으며, 최종적으로는 GPU 서버에서 ACT 정책 추론을 수행하고 Jetson Orin Nano에서는 카메라 영상 및 로봇 상태 수집과 실제 로봇 제어를 담당하는 분산 구조로 변경하였다.

학습된 ACT 정책은 Hugging Face Hub를 통해 모델을 관리하였으며, 추론 서버에서는 해당 정책을 불러와 카메라 영상과 로봇 상태를 입력으로 ACT 추론을 수행하도록 구성하였다. 추론 결과로 생성된 로봇 관절 제어 명령은 네트워크를 통해 Jetson Orin Nano로 전달되며, Jetson에서는 전달받은 명령을 SO-101 팔로워 암에 적용하도록 구성하였다. 학습, 정책 추론, 카메라 처리 및 실제 로봇 제어를 각각의 환경에 적합하게 분리하였다.

3.2 Teleoperation 기반 데이터 수집 및 데이터셋 구축

3.2.1 Pick & Stack 시연 데이터 수집 절차

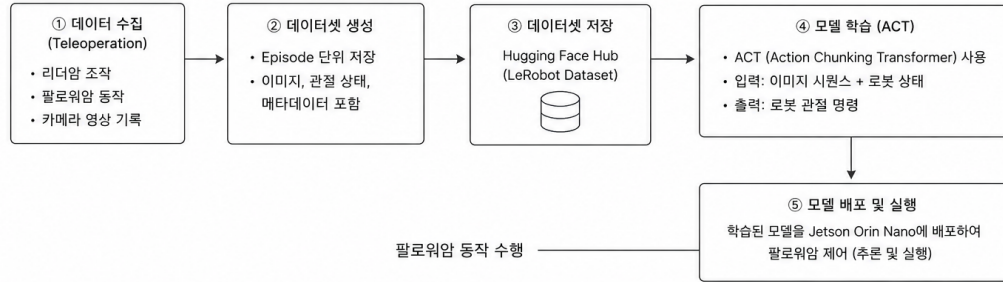


그림 3.6: Teleoperation 기반 데이터 수집 및 ACT 모델 학습 과정

본 연구의 실험 과제는 작업 공간에 배치된 블록을 로봇팔로 집어 지정된 영역에 순차적으로 적재하는 Pick & Stack 작업으로 정의하였다. 데이터 수집은 사용자가 리더 암을 직접 조작하여 팔로워 암이 동일한 동작을 수행하도록 하는 텔레오퍼레이션(Teleoperation) 방식으로 진행하였다. 각 시연 과정에서는 로봇의 관절 상태와 카메라 영상을 동시에 기록하였다. 로봇 상태 데이터는 6개 관절의 상태를 포함하며, 영상 데이터는 작업 공간을 관찰하는 상단 카메라와 그리퍼 주변을 관찰하는 손목 카메라에서 수집하였다.

수집된 하나의 작업 수행 단위를 하나의 에피소드(episode)로 구성하였다. 각 에피소드는 작업 시작부터 블록을 집고 지정된 영역에 적재하는 과정까지의 연속적인 관측 및 행동 데이터로 기록되며, 이를 반복적으로 수집하여 ACT 정책 학습에 사용할 시연 데이터셋을 구성하였다. 특정 위치나 특정 접근 방향에 대한 정책의 과도한 의존을 줄이고자 동일한 작업을 반복적으로 수행하는 것에 그치지 않고, 블록의 위치 및 접근 방향을 변화시켜 다양한 상황에서 로봇이 블록을 집는 동작을 학습할 수 있도록 하였다.

3.2.2 외부 오픈소스 데이터셋 호환성 검토 및 자체 환경 동기화

본 과제에서는 초기 데이터 수집의 물리적 부담을 줄이고 데이터의 다양성을 확보하기 위해, ALOHA 프로젝트나 LeRobot 커뮤니티에서 제공하는 유사 오픈소스 데이터셋 (svla_so100)의 활용을 적극 검토하였다.[5] 그러나 외부 데이터셋을 본 과제의 환경에 적용하기 위해 데이터 호환성을 분석한 결과, 다음과 같은 치명적인 도메인 차이(Domain Gap)

및 물리적 제약이 확인되었다.

- **환경 및 카메라 화각의 불일치:** 외부 데이터셋의 작업 환경과 본 과제의 환경(20cm x 10cm 지정 영역 등) 간의 미세한 카메라 뷰 차이는 데이터셋 병합 시 시각적 관측(Observation) 데이터의 일관성을 크게 훼손하는 요인으로 작용하였다.
- **기구학적 오차 및 캘리브레이션 불일치:** 타 연구 환경과의 Z축 단차 및 조인트 영점(Zero-point) 불일치로 인해, 외부 데이터를 그대로 모방할 경우 로봇이 작업대 바닥과 물리적으로 강하게 충돌할 위험성이 도출되었다. 이는 데이터 궤적의 오류를 넘어 로봇 하드웨어의 손상을 유발할 수 있는 치명적인 불량 데이터로 작용할 우려가 컸다.
- **데이터 분포의 이질성:** 외부 데이터셋에 포함된 로봇의 궤적 패턴과 상태 공간(State Space)이 실제 테스트 환경과 상이하여, 이종(Heterogeneous) 데이터셋을 단순 혼합할 경우 특정 궤적이나 영역에서 데이터 분포가 심하게 왜곡되는 편향성이 발생함을 확인하였다.

결과적으로, 미세한 카메라 뷰의 차이와 캘리브레이션 제약이 양질의 궤적 데이터 확보에 악영향을 미친다는 점을 확인하였다. 이에 따라 외부 데이터셋 의존을 배제하고, 본 과제의 실제 물리적 환경에 완벽히 동기화된 자체 시연 데이터만을 엄격하게 수집하여 데이터 파이프라인을 재구성하는 방향으로 수집 전략을 확립하였다.

3.2.3 작업 연속성을 고려한 시연 에피소드(Episode) 설계 및 수집 전략

ACT 기반 모방학습을 적용하기 위해서는 로봇의 관측 상태와 이에 대응하는 행동을 시연 데이터의 형태로 수집해야 하며, 이때 하나의 에피소드가 어떠한 작업 범위를 포함하도록 구성할 것인지 결정하는 과정이 필요하였다. 본 연구의 최종 작업은 작업 공간에 배치된 5개의 블록을 순차적으로 파지하고 지정된 영역에 적재하는 Pick & Stack 작업이므로, 단순히 개별 블록을 집는 동작만 반복적으로 수집하는 방식과 실제 전체 작업 과정을 하나의 연속적인 에피소드로 수집하는 방식 중 어떤 방법이 적합한지 검토하였다.

초기에는 데이터 수집의 효율성과 기본적인 Pick 동작의 반복 학습을 우선적으로 고려하여, 하나의 블록을 집어 목표 영역에 배치하는 과정을 하나의 독립적인 에피소드로 구성하는 방식을 적용하였다. 이 방식에서는 하나의 블록을 파지하고 이동하여 지정된 영역에 배치한 후 에피소드를 종료하고, 다음 에피소드에서는 다시 초기 상태에서 새로운 블록을 파지하는 방식으로 데이터를 수집하였다.

이러한 단일 블록 에피소드 방식은 에피소드의 길이가 짧아 동일한 시간 동안 많은 에피소드를 확보할 수 있다는 장점이 있었다. 또한 하나의 에피소드에 하나의 Pick & Place

동작만 포함되므로 블록에 접근하고 파지하는 과정에서 필요한 관절 움직임과 그리퍼 조작을 반복적으로 확보할 수 있었다. 따라서 초기 단계에서는 로봇이 기본적인 블록 파지 및 배치 동작을 학습하기 위한 데이터 수집 방식으로 적합하다고 판단하였다.

이에 따라 초기 시범 데이터로 약 10초 길이의 단일 블록 에피소드 약 200개를 수집하였다. 수집된 데이터를 학습하고 테스트하는 과정에서 각 에피소드의 시작 및 종료 상태와 로봇의 관절 궤적을 중심으로 데이터가 실제 작업 과정을 충분히 포함하고 있는지를 분석하였다.

분석 결과, 단일 블록 에피소드 방식에서는 하나의 블록을 처리한 이후 에피소드가 종료되고 다음 에피소드가 다시 초기 상태에서 시작하기 때문에, 실제 Pick & Stack 작업에서 발생하는 블록 간 연속적인 상태 전이가 충분히 포함되지 않는 문제가 확인되었다. 실제 작업에서는 첫 번째 블록을 적재한 이후 로봇이 위치한 상태가 두 번째 블록에 접근하기 위한 새로운 시작 상태가 되며, 이후에도 이전 블록을 처리한 결과가 다음 블록을 처리하는 과정에 영향을 미친다.

반면 단일 블록 에피소드에서는 이러한 연속적인 상태 전이가 에피소드 경계에서 단절된다. 결과적으로 실제 작업에서는 존재하는 “이전 블록 적재 상태 → 다음 블록 접근 상태”와 같은 전이 관계가 학습 결과에서 충분하게 나타나지 않으며, 각 에피소드가 초기 상태에서 시작하는 구조가 반복적으로 나타났다. 이를 통해 단순히 Pick 에피소드 개수를 증가시키는 것만으로는 본 연구의 최종 작업에서 요구되는 연속적인 행동 관계와 상태 전이를 충분히 확보하기 어렵다는 점을 확인하였다.

이러한 문제를 해결하기 위해 이후 데이터 수집에서는 하나의 블록을 처리할 때마다 에피소드를 종료하는 대신, 작업 공간에 배치된 5개의 블록을 하나의 에피소드에서 연속적으로 처리하도록 수집 단위를 변경하였다. 변경된 방식에서는 약 40초 동안 블록에 접근하고, 파지하고, 목표 영역으로 이동하여 적재한 뒤, 다시 다음 블록으로 이동하는 전체 과정을 하나의 에피소드로 기록하였다. 40초로 정한 이유는 데이터셋을 직접 만들어보는 여러 시도 결과 가장 안정적이고 신속하게 하나의 에피소드를 완성하는데 걸린 시간이 평균 약 40초였기 때문이다.

전체 블록 연속 수집 방식에서는 이전 블록을 적재한 이후의 로봇 상태가 다음 블록을 파지하기 위한 초기 상태로 자연스럽게 연결된다. 따라서 단일 블록 방식에서는 에피소드 경계로 인해 분리되었던 블록 간 이동과 상태 전이를 하나의 연속적인 데이터로 확보할 수 있었다. 또한 블록의 위치와 접근 방향이 달라지는 경우 블록 간 이동 궤적과 로봇의 관절 상태 역시 변화하므로, 하나의 에피소드 안에서도 서로 다른 관측 및 행동 패턴을 포함할 수 있었다.

그러나 전체 블록을 하나의 에피소드로 구성하는 것만으로는 데이터가 특정 블록 위치나 특정 접근 방향에 편중되는 문제를 완전히 해결할 수 없었다. 예를 들어 블록이 매번 유사한

위치에 배치되는 경우, 에피소드의 길이는 충분하더라도 로봇이 경험하는 시각적 관측과 관절 상태의 범위가 제한될 수 있다. 따라서 이후에는 에피소드의 연속성을 유지하면서 동시에 블록의 초기 위치를 변화시키는 방향으로 데이터 수집 전략을 추가적으로 확장하였다.

이에 따라 작업 공간 내 블록의 위치 조건을 고정 위치, 좌측 편향, 우측 편향, 랜덤 위치 및 특이 위치로 구분하고 각각의 조건에 대한 시연 데이터를 추가적으로 수집하였다. 고정 위치 데이터는 기본적인 작업 수행 패턴을 확보하기 위한 기준 데이터로 사용하였으며, 좌측 및 우측 편향 데이터는 작업 공간의 특정 방향에 블록이 위치했을 때 발생하는 접근 궤적과 관절 상태 변화를 학습 데이터에 포함하기 위해 구성하였다. 랜덤 위치 데이터는 블록의 위치를 작업 공간 전반으로 변화시켜 특정 위치에 대한 정책의 의존성을 줄이기 위해 구성하였으며, 특이 위치 데이터는 구석이나 블록 간 인접 배치와 같이 일반적인 배치 조건에서 벗어난 상황에 대한 대응 데이터를 확보하기 위해 구성하였다.

이와 같이 데이터 수집 과정을 단계적으로 변경하면서 본 연구에서는 데이터의 양뿐만 아니라 데이터가 포함하는 상태 공간과 상태 전이의 다양성이 중요하다는 점을 확인하였다. 초기 단일 블록 에피소드에서는 개별 Pick 동작을 반복적으로 확보할 수 있었으나 실제 연속 작업에서 발생하는 상태 전이를 충분히 표현하기 어려웠으며, 전체 블록 연속 에피소드로 변경한 이후에는 실제 작업과 유사한 연속적인 행동 궤적을 확보할 수 있었다. 이후 위치 조건을 다양화함으로써 작업 공간 내에서 정책이 경험할 수 있는 관측 및 행동 분포를 추가적으로 확장하였다.

따라서 최종적으로 본 연구에서는 5개 블록을 연속적으로 처리하는 전체 작업 단위를 하나의 에피소드로 정의하고, 각 에피소드의 초기 블록 배치를 변화시키는 방식으로 시연 데이터를 수집하였다. 이러한 데이터 수집 구조는 이후 위치 조건별 데이터셋을 개별적으로 관리하고 필요한 데이터를 통합하는 과정뿐만 아니라, 실제 추론 과정에서 발생한 실패 상태를 추가적인 전문가 시연으로 보완하는 DAgger 기반 데이터 보강 과정으로 확장할 수 있는 기반으로 활용하였다.

본 연구의 최종 목표는 다수의 블록을 연속적으로 파지하고 순차적으로 적재하는 임무를 수행하는 것이다. 이를 위해 로봇의 연속적인 동작을 가장 효과적으로 담아낼 수 있는 최적의 시연 에피소드(Episode) 단위를 설계하는 과정이 필요하였다. 초기 데이터 수집 설계 단계에서는 데이터 구축의 효율성을 고려하여 두 가지 수집 방식을 비교 및 검토하였다.

1. 단일 블록 에피소드 분할 수집의 구조적 한계

초기에는 데이터 확보 속도를 높이고 파지(Grasp) 동작을 집중적으로 수집하기 위해, 5개의 블록 중 단 1개만을 목표 영역으로 이동시키는 약 10초 분량의 '단일 블록 수집 방식'을 검토하였다. 시범적으로 수집된 200개의 단일 에피소드 데이터를 분석한 결과, 해당 방식은 '상태 전이(State Transition)'의 다양성을 데이터에 담아내지 못하는 구조적 한계를 보였다. 실제

연속 작업에서는 이전 블록을 내려놓은 로봇의 마지막 위치가 다음 블록을 집기 위한 새로운 시작 위치가 되어야 한다. 그러나 단일 에피소드 방식은 짧은 궤적 특성상 대기 위치(Home position) 등 제한된 시작점에서만 출발하는 궤적이 지배적으로 기록되어, 시작 지점과 대상 블록 간의 상대적 위치 다양성이 훼손되고 데이터 분포가 편향되는 문제가 발생하였다.

2. 전체 블록 연속 수집 방식 채택 및 데이터셋 고도화

데이터 분포의 편향성을 해결하고 작업의 연속성을 확보하기 위해, 본 연구에서는 5개의 블록을 모두 처리하는 약 40초 분량의 '**전체 블록 연속 수집 방식**'을 최종 에피소드 단위로 채택하였다. 이 방식을 통해 작업의 종료 시점이 다음 작업의 시작 시점과 자연스럽게 이어지는 연속적인 궤적 데이터를 확보할 수 있었다.

더불어, 긴 에피소드 수집 과정에서 발생할 수 있는 데이터 획일화 문제를 방지하기 위해 다음과 같은 수집 고도화 전략을 적용하였다.

- **초기 상태(Initial State)의 무작위성 강화:** 블록의 배치를 특정 구역에 국한하지 않고, 좌/우측으로 심하게 편향시키거나 넓게 흩어놓는 등 시각적 관측 다양성을 극대화하였다.
- **동적 전이 궤적 확보:** 정해진 순서 없이 무작위 방향에서 블록에 접근하도록 시연하고, 이전 블록을 놓은 위치에서 바로 다음 블록으로 이동하는 다양한 상태 전이 궤적을 중점적으로 수집하였다.
- **오류 복구(Recovery) 시연 포함:** 완벽한 성공 궤적뿐만 아니라, 블록 파지에 실패하거나 떨어뜨린 상태에서 대상을 다시 추적하여 집어 드는 복구 과정을 시연에 의도적으로 포함시켜 데이터셋의 강건성을 높였다.

이러한 에피소드 설계의 전환을 통해, 태스크의 본질적인 연속성을 시연 데이터에 온전히 반영할 수 있는 기반을 마련하였다.

3.2.4 블록 위치 조건별 데이터 수집

앞선 에피소드 설계 과정에서는 실제 Pick & Stack 작업의 연속성을 반영하기 위해 5개의 블록을 하나의 에피소드에서 연속적으로 처리하는 방식을 채택하였다. 이후에는 하나의 에피소드 내부에서 로봇이 다양한 작업 상태를 경험할 수 있도록 블록의 초기 배치 위치를 변화시키는 수집 전략을 설계하였다. 이는 동일한 작업을 동일한 위치에서 반복하여 수집할 경우 시연 데이터의 관측 및 행동 분포가 특정 위치에 편중될 수 있기 때문이다.

특히 ACT 기반 모방학습에서는 카메라 영상과 로봇 상태를 입력으로 사용하여 시연 데이터에서 관측되는 상황과 행동의 관계를 학습하기 때문에, 학습 데이터에 포함되지 않은

위치에서 블록이 나타나는 경우 학습된 정책이 적절한 접근 및 파지 행동을 생성하지 못할 가능성이 있다. 따라서 본 연구에서는 단순히 전체 에피소드 수를 증가시키는 것보다, 실제 작업 공간에서 블록이 위치할 수 있는 공간적 조건을 사전에 구분하고 각 조건에 대한 데이터를 확보하는 방향으로 데이터 수집 정책을 구성하였다.

초기 데이터 수집에서는 블록을 사전에 정의된 고정 위치에 배치하였다. 고정 위치 데이터는 로봇이 기본적인 블록 접근, 파지, 이동 및 적재 동작을 안정적으로 수행하는지를 확인하기 위한 기준 데이터로 활용하였다. 또한 이후 다양한 위치 조건의 데이터를 추가할 때 기본적인 작업 수행 패턴과 비교할 수 있는 기준으로 사용하기 위해 일정한 위치에서 반복적으로 데이터를 확보하였다.

그러나 고정 위치에서 수집한 데이터만으로는 블록의 위치가 변화했을 때 발생하는 관측 및 행동의 변화를 충분히 포함하기 어렵다는 문제가 있었다. 블록이 항상 동일한 위치에 존재하면 상단 카메라에서 관측되는 블록의 위치가 제한되고, 블록에 접근하기 위한 로봇의 관절 궤적 역시 유사한 형태로 반복된다. 따라서 이러한 데이터를 중심으로 학습할 경우 실제 추론 과정에서 블록의 위치가 달라졌을 때 학습 데이터와 실제 입력 사이에 차이가 발생할 수 있다고 판단하였다.

이에 따라 먼저 블록이 작업 공간의 특정 방향으로 편향되는 상황을 별도로 구성하였다. 좌측 편향 및 우측 편향 조건은 블록이 한쪽 영역에 집중적으로 배치되는 상황을 의미하며, 이를 통해 고정 위치 데이터에서 부족했던 방향별 접근 궤적을 확보하고자 하였다. 좌측과 우측의 데이터를 각각 수집함으로써 로봇이 한쪽 방향에서만 블록에 접근하는 패턴을 학습하는 것을 방지하고, 작업 공간의 좌우 방향에 따른 관절 상태 및 카메라 관측의 변화를 데이터에 포함하였다.

이후에는 단순히 좌측 또는 우측으로 편향된 상황뿐만 아니라 작업 공간 전체에서 발생할 수 있는 다양한 위치 분포를 확보하기 위해 랜덤 위치 데이터를 구성하였다. 다만 본 연구에서 사용하는 랜덤 위치 데이터는 블록을 아무런 기준 없이 완전히 무작위로 배치하는 방식으로 구성하지 않았다. 완전한 무작위 배치만 사용할 경우 사람이 직접 블록을 놓고 데이터셋 녹화를 시작하기 때문에 본인도 모르게 가장 익숙한 구역에 블록을 놓아 특정 위치 분포가 우연히 과도하게 포함되어 특정 조건이 충분히 확보되지 않을 가능성을 방지하고자, 작업 공간 내에서 발생할 수 있는 주요 위치 분포를 사전에 정의하고 각 조건을 일정한 수량으로 수집하였다.

먼저 전체 5개 블록의 좌우 분포를 기준으로 네 가지 조건을 구성하였다. 좌측에 4개, 우측에 1개의 블록이 위치하는 조건, 좌측에 3개, 우측에 2개의 블록이 위치하는 조건, 좌측에 2개, 우측에 3개의 블록이 위치하는 조건, 좌측에 1개, 우측에 4개의 블록이 위치하는 조건 각각 20개 에피소드씩 수집하였다. 이를 통해 블록이 좌측 또는 우측으로 강하게 편향되는 상황부터 반대 방향으로 편향되는 상황까지 다양한 좌우 분포를 균등하게 확보하였다.

또한 블록의 좌우 분포만으로는 작업 공간의 상하 방향에 따른 위치 변화가 충분히 반영되지 않을 수 있으므로 블록이 작업 공간의 상측에 집중되는 조건, 하측에 집중되는 조건

또한 각각 20개 에피소드씩 추가하였다. 이를 통해 상단 카메라 영상에서 블록의 관측 위치가 상하 방향으로 변화하는 상황과 이에 대응하여 로봇이 블록에 접근하기 위해 수행하는 관절 움직임의 변화를 포함하였다.

마지막으로 일반적인 위치 분포보다 작업 난도가 높을 수 있는 상황을 확보하기 위해 작업 공간의 경계 주변 및 블록 간 인접 배치 조건을 별도로 구성하였다. 해당 조건에서는 블록이 작업 공간의 경계에 가깝게 배치되거나 여러 블록이 서로 붙어 있는 상황을 포함하였다. 이러한 배치는 특정 블록에 접근하는 과정에서 주변 블록과의 간섭을 고려해야 하거나 접근 가능한 방향이 제한될 수 있으므로, 일반적인 위치에서 수집된 데이터와 다른 행동 궤적을 발생시킬 수 있다. 따라서 해당 조건을 별도로 수집하여 학습 데이터에 포함하였다.

위와 같은 기준에 따라 랜덤 위치 데이터는 좌우 분포 조건 4개에 대해 각각 20개씩 총 80개, 상측 집중 조건 20개, 하측 집중 조건 20개, 경계 주변 및 블록 인접 조건 40개로 구성하여 총 160개의 세부 조건 기반 데이터 수집을 계획하였다.

표 3.1: 랜덤 위치 데이터의 세부 구성

세부 위치 조건	블록 배치 기준	에피소드 수
좌측 편향 1	좌 4 / 우 1	20
좌측 편향 2	좌 3 / 우 2	20
좌우 분포 3	좌 2 / 우 3	20
우측 편향 4	좌 1 / 우 4	20
상측 집중	작업 공간 상측 배치	20
하측 집중	작업 공간 하측 배치	20
경계 및 인접	경계 주변 및 블록 간 인접 배치	40
합계		160

최종적으로 본 연구의 위치 조건별 데이터 수집은 고정된 위치에서 동일한 작업을 반복하는 방식에서 시작하여 좌우 편향, 상하 위치 변화, 랜덤 분포 및 경계·인접 상황을 단계적으로 추가하는 방식으로 확장하였다. 이를 통해 단일 위치에 집중된 데이터 분포를 완화하고, 작업 공간 내에서 블록의 위치와 주변 블록과의 관계가 변화하는 다양한 상태를 시연 데이터에 포함하였다. 또한 위치 조건별 데이터를 별도로 관리함으로써 이후 모델 학습 결과에 따라

특정 조건의 데이터를 추가하거나 제거할 수 있도록 데이터셋을 구성하였다.

한편 DAgger 교정 데이터는 초기 위치 조건 데이터를 충분히 확보한 이후 실제 학습 정책의 추론 과정에서 발생하는 실패 상태를 대상으로 수집하였다. 따라서 위치 조건별 데이터 수집은 학습 이전에 예상 가능한 작업 공간의 상태 다양성을 확보하는 과정이며, DAgger 데이터는 학습된 정책이 실제 추론 과정에서 취약성을 보인 상태를 추가적으로 보완하는 과정으로 구분하였다.

3.2.5 블록 탐색 및 선택 정책 결정 과정

본 연구에서는 하나의 작업 공간에 여러 개의 블록이 동시에 존재하는 Pick & Stack 작업의 특성을 고려하여, 로봇이 여러 블록 중 어떤 대상을 우선적으로 선택할 것인지에 대한 기준을 설정하였다. 블록 선택 방식은 데이터 수집 과정에서 반복적으로 나타나는 행동 패턴을 결정하기 때문에, 단순히 작업 수행 시간을 줄이는 기준을 사용하는 것보다 실제 평가 환경에서 활용할 수 있는 정보와 다양한 작업 상황을 학습 데이터에 포함할 수 있는지를 우선적으로 고려하였다. 이에 따라 블록의 색상 기반 선택, 로봇과 블록 사이의 거리 기반 선택 및 공간적 위치와 주변 환경을 고려한 선택 방식을 비교·검토하고 최종 수집 기준을 결정하였다.

먼저 **블록의 색상을 기준으로 선택하는 방식은 데이터 수집 및 학습 과정에서 사용하지 않았다**. 본 과제의 실제 평가에서는 여러 블록이 동일한 색상으로 구성될 수 있기 때문에, 학습 데이터에서 특정 색상의 블록을 우선적으로 선택하도록 구성하는 것은 평가 환경에서 활용할 수 없는 정보에 의존하는 결과를 만들 수 있다. 예를 들어 학습 데이터에서 특정 색상의 블록이 반복적으로 먼저 선택되는 경우, 모델이 블록의 공간적 위치나 주변 환경뿐만 아니라 색상 자체를 블록 선택의 단서로 활용할 가능성이 있다. 그러나 평가 환경에서 모든 블록의 색상이 동일해지면 이러한 색상 기반 단서가 더 이상 구별 정보를 제공하지 못한다. 따라서 본 연구에서는 색상을 블록 선택의 결정 기준에서 제외하고, 실제 평가 환경에서도 관측 및 활용 가능한 공간적 정보에 집중하도록 데이터 수집 방향을 설정하였다.

다음으로 **로봇에서 가장 가까운 블록을 우선적으로 선택하는 거리 기반 방식**을 검토하였다. 가장 가까운 블록부터 선택하는 방식은 개별 동작에서 로봇의 이동 거리를 줄이고 작업 시간을 단축할 수 있다는 장점이 있다. 그러나 본 연구의 데이터셋 구축 목적은 각 에피소드의 이동 거리를 최소화하는 것이 아니라, 서로 다른 블록 배치와 접근 상황에 대응할 수 있는 다양한 관측 및 행동 데이터를 확보하는 데 있다. 모든 에피소드에서 현재 로봇 위치와 가장 가까운 블록만을 반복적으로 선택할 경우, 블록 선택 순서가 로봇의 현재 위치에 의해 일정한 형태로 결정될 가능성이 높으며, 결과적으로 데이터셋에 포함되는 접근 방향과 상태 전이가 제한될 수 있다.

특히 본 연구에서는 블록의 위치에 따른 정책의 일반화 성능을 확보하기 위하여 좌측 및

우측 편향, 상·하단 배치, 경계 주변 배치 및 블록 간 인접 배치 등의 다양한 조건을 별도로 구성하였다. 이러한 조건에서 단순 거리 기준만을 적용할 경우, 위치 조건을 다양하게 구성하더라도 실제 시연 과정에서는 항상 가까운 블록을 우선적으로 선택하는 유사한 행동 패턴이 반복될 수 있다. 이는 데이터셋의 공간적 다양성을 실제 행동 다양성으로 충분히 연결하지 못하는 문제가 발생할 수 있다. 따라서 본 연구에서는 **이동 거리의 최소화보다 블록 선택 및 접근 상황의 다양성을 확보하는 것을 우선적인 데이터 수집 기준으로 설정하였다.**

이에 따라 본 연구에서는 특정 색상이나 단순한 거리 순서를 고정적으로 적용하지 않고, **블록의 공간적 위치와 주변 블록의 배치 상태를 고려하여 선택하는 방식**을 적용하였다. 여기서 랜덤 선택은 블록을 아무런 기준 없이 무작위로 선택한다는 의미가 아니라, 특정 블록이 항상 동일한 우선순위를 갖지 않도록 하면서 다양한 공간적 상황을 시연 데이터에 포함시키기 위한 방법으로 정의하였다. 즉, 데이터 수집 과정에서 블록의 위치 및 주변 환경을 확인하고, 특정 위치의 블록만 반복적으로 선택하지 않도록 선택 상황을 다양화하였다.

이러한 목적을 달성하기 위하여 랜덤 위치 데이터 역시 완전한 무작위 방식으로 생성하지 않고, 작업 공간 내에서 발생할 수 있는 대표적인 공간 분포를 사전에 정의하여 구성하였다. 구체적으로 좌측 블록 4개와 우측 블록 1개가 배치되는 조건, 좌측 3개와 우측 2개가 배치되는 조건, 좌측 2개와 우측 3개가 배치되는 조건, 좌측 1개와 우측 4개가 배치되는 조건을 각각 20개 에피소드씩 수집하였다. 또한 작업 공간의 위쪽과 아래쪽에 블록이 집중되는 조건을 각각 20개씩 구성하였으며, 블록이 작업 영역의 경계 주변에 위치하거나 서로 인접하여 배치되는 상황은 40개 에피소드로 별도 구성하였다. 이를 통해 특정 위치가 우연히 반복되는 단순 무작위 데이터가 아니라, 실제 작업에서 발생 가능한 공간적 편향과 난이도 높은 배치 상황을 의도적으로 포함하는 방향으로 데이터셋을 구성하였다.

또한 Task 1과 Task 2는 작업의 목적이 서로 다르므로 블록 선택 이후의 처리 방식에도 차이를 두었다. Task 1에서는 **5개의 블록을 지정된 목표 영역 내부에 모두 배치하는 것을 목표로 하였으며, 목표 영역 내의 공간을 효율적으로 사용하기 위하여 W 형태의 순서로 블록을 배치하도록 시연하였다.** 이는 먼저 배치된 블록으로 인해 이후 블록을 놓을 수 있는 공간이 제한되는 상황을 줄이고, 5개의 블록을 목표 영역 내부에 안정적으로 배치하기 위한 기준이다. 따라서 Task 1에서는 블록의 선택 과정에서 다양한 위치의 블록을 경험하도록 하면서도, 최종적인 목표 영역 내 배치 과정에서는 W 형태의 일관된 배치 전략을 적용하였다.

반면 Task 2에서는 이미 적재된 블록 위에 다음 블록을 순차적으로 올려야 하므로, 단순한 이동 거리뿐만 아니라 주변 블록 및 기존 Stack과의 간섭 가능성을 함께 고려하였다. 따라서 블록 간 거리가 매우 가깝거나 서로 붙어 있는 경우에는 단순히 가장 가까운 블록을 선택하기보다 이동 과정에서 다른 블록과 충돌할 가능성이 낮은지를 함께 고려하여 시연하였다. 이를 통해 Task 2에서는 블록 선택, 이동 및 적재 과정에서 발생하는 상태 전이와 Stack의 안정성을

동시에 고려할 수 있도록 하였다.

결과적으로 본 연구의 블록 선택 정책은 **색상과 같은 평가 환경에 따라 사라질 수 있는 정보나 단순한 거리 최소화에 의존하지 않고, 공간적 위치와 주변 환경을 중심으로 다양한 선택 및 접근 상황을 확보하는 것**을 기본 원칙으로 설정하였다. 이를 통해 데이터셋의 목표를 단순한 작업 시간 단축이 아닌 다양한 작업 조건에 대한 관측 및 행동 패턴 확보로 정의하였으며, 이후 ACT 정책이 특정 블록 위치나 특정 선택 순서에 과도하게 의존하는 것을 줄이고 다양한 작업 공간 조건에 대응할 수 있도록 데이터 수집 전략을 구성하였다.

3.2.6 시연 데이터 수집 매뉴얼

앞서 정의한 블록 탐색 및 선택 정책을 실제 Teleoperation 기반 시연 데이터 수집 과정에 적용하기 위해, 시연자가 로봇팔을 조작할 때 준수해야 하는 기본적인 수집 매뉴얼을 정의하였다. Teleoperation 데이터는 조작자의 판단과 움직임이 그대로 로봇의 행동 데이터로 기록되기 때문에, 동일한 작업 조건에서도 파지 위치, 이동 높이, 이동 경로 및 Release 방식에 따라 서로 다른 행동 궤적이 생성될 수 있다. 따라서 본 연구에서는 다양한 블록 위치에 대한 시연 데이터의 다양성은 유지하면서도, 불필요한 조작 차이와 반복적인 작업 실패가 데이터에 포함되는 것을 줄이기 위한 공통적인 행동 기준을 설정하였다.

첫 번째로, **블록을 파지할 때에는 그리퍼를 블록의 높이에 맞추어 충분히 하강시킨 후 파지하도록 하였다.** 그리퍼가 블록의 상단에 충분히 접근하지 않은 상태에서 파지할 경우 블록을 안정적으로 잡지 못하거나 파지 과정에서 블록이 움직일 수 있기 때문이다. 이에 따라 시연자는 블록의 위치를 확인한 후 그리퍼를 가능한 낮은 위치까지 이동시키고, 블록을 충분히 감싼 상태에서 그리퍼를 닫도록 하였다. 이를 통해 파지 직후 블록이 기울어지거나 떨어지는 상황을 최소화하고, 이후 이동 과정에서 안정적인 상태가 유지되도록 하였다.

두 번째로, **블록을 파지한 직후에는 수평 방향으로 바로 이동하지 않고 먼저 블록을 위쪽으로 들어 올리도록 하였다.** 작업 공간에는 여러 블록이 동시에 배치되어 있기 때문에 낮은 높이에서 블록을 이동할 경우 주변 블록이나 작업대와 충돌할 가능성이 존재한다. 특히 블록 간 거리가 가깝거나 서로 인접한 경우에는 이러한 충돌 가능성이 더욱 증가한다. 따라서 블록을 안정적으로 파지한 이후 충분한 높이까지 들어 올린 다음 목표 위치 방향으로 이동하도록 하여 주변 물체와의 간섭을 줄였다.

세 번째로, **블록을 이동할 때에는 주변 블록과의 충돌을 최소화할 수 있는 경로를 선택하도록 하였다.** 단순히 로봇에서 가장 가까운 블록을 선택하여 이동 거리를 최소화하는 것을 데이터 수집의 최우선 기준으로 설정하지 않았다. 대신 현재 로봇팔의 위치와 블록의 위치, 주변에 존재하는 다른 블록 및 이미 적재된 Stack의 위치를 함께 고려하여 이동 경로를 결정하도록 하였다. 이를 통해 불필요하게 다른 블록을 통과하거나 기존에 배치된 블록을 건드리는

상황을 줄이고, 안정적인 작업 궤적을 확보하고자 하였다.

네 번째로, **목표 위치에 블록을 적재할 때에는 기존 블록이나 작업대에 그리퍼가 직접 접촉한 상태에서 Release하지 않도록 하였다.** 특히 Task 2와 같이 블록을 여러 층으로 적재하는 경우에는 Release 과정에서 발생하는 작은 충격이나 접촉만으로도 기존 Stack의 위치가 변할 수 있으며, 이러한 위치 변화가 누적될 경우 최종적으로 Stack이 기울어지거나 무너질 수 있다. 따라서 목표 위치와 블록의 방향을 충분히 정렬한 후 기존에 적재된 블록으로부터 약간 떨어진 위쪽 위치에서 그리퍼를 열어 블록을 Release하도록 하였다. 이를 통해 적재 과정에서 기존 Stack에 가해지는 불필요한 접촉을 줄이고 안정적인 적재 궤적을 확보하였다.

다섯 번째로, **Task 1에서는 목표 영역 내의 공간을 효율적으로 사용하기 위하여 블록을 W 형태의 순서로 배치하도록 하였다.** Task 1은 5개의 블록을 지정된 목표 영역 내부에 모두 배치하는 것을 목표로 하므로, 개별 블록을 성공적으로 이동시키는 것뿐만 아니라 이후 블록을 배치할 수 있는 공간을 확보하는 것이 중요하다. 이에 따라 목표 영역의 공간을 고려하여 5개의 블록을 W 형태로 순차 배치하는 기준을 적용하였다. 이를 통해 먼저 배치된 블록이 이후 블록의 배치 공간을 과도하게 차지하는 상황을 줄이고, 전체 블록을 목표 영역 내부에 안정적으로 배치할 수 있도록 하였다.

여섯 번째로, **Task 2에서는 이미 적재된 블록과 다음에 이동할 블록 사이의 간섭을 고려하여 시연하도록 하였다.** Task 2는 하나의 블록을 목표 영역에 놓는 것으로 작업이 종료되는 Task 1과 달리, 이전 블록 위에 다음 블록을 순차적으로 적재하는 과정이 반복된다. 따라서 각 블록을 이동시키는 과정에서 기존 Stack과의 충돌을 방지하고, 블록을 적재할 때 Stack이 흔들리지 않도록 하는 것이 중요하다. 이에 따라 주변 공간이 충분히 확보된 블록을 우선적으로 처리하고, 블록 간 인접도가 높은 경우에는 이동 경로를 조정하여 다른 블록과의 충돌을 최소화하도록 하였다.

마지막으로, 이러한 매뉴얼은 모든 시연 데이터를 동일한 궤적으로 만들기 위한 것이 아니라 **불필요한 실패와 조작 편차를 줄이기 위한 최소한의 공통 기준**으로 적용하였다. 블록의 초기 배치와 선택 상황은 앞서 정의한 위치 조건에 따라 다양하게 구성하여 데이터의 다양성을 확보하되, 파지, 들어 올리기, 이동 및 Release 과정에서는 공통적인 안전 기준을 적용하였다. 따라서 데이터셋에는 서로 다른 블록 위치와 접근 방향에 대한 다양한 행동 궤적을 포함하면서도, 파지 실패나 주변 블록과의 충돌, Stack 붕괴와 같이 학습에 불필요한 오류가 반복적으로 포함되는 것을 방지하고자 하였다.

결과적으로 본 연구에서는 **블록 선택 단계에서는 특정 색상이나 단순한 거리 순서에 의존하지 않고 공간적 위치와 주변 환경을 고려하였으며, 실제 Teleoperation 시연 단계에서는 안정적인 파지와 이동 및 적재를 위한 공통 매뉴얼을 적용하였다.** 이를 통해 데이터의 다양성과 작업 수행의 안정성을 동시에 확보할 수 있도록 시연 데이터 수집 과정을 구성하였다.

3.2.7 데이터 품질 관리 및 비정상 에피소드 검증·제거

Teleoperation 기반으로 시연 데이터를 수집하는 과정에서는 로봇의 정상적인 작업 수행 과정뿐만 아니라 조작자의 실수, 로봇 및 카메라의 일시적인 오류, 작업 실패 등의 비정상적인 상황이 함께 기록될 수 있다. 본 연구에서 사용하는 ACT는 시연 데이터에 포함된 관측과 행동의 관계를 학습하여 이후의 행동을 예측하는 방식이므로, 정상적인 작업 수행을 나타내지 않는 에피소드가 학습 데이터에 과도하게 포함될 경우 잘못된 행동 패턴까지 정책이 학습할 가능성이 있다.

따라서 본 연구에서는 데이터 수집 이후 모든 에피소드를 무조건 학습에 사용하는 것이 아니라, 각 에피소드가 정상적인 Pick & Stack 수행 과정을 충분히 포함하고 있는지를 확인하는 데이터 품질 관리 과정을 수행하였다. 데이터 검증은 크게 시스템 및 기록 오류, 로봇 궤적 및 조작 오류, 파지 과정의 오류, Task 수행 실패 및 적재 실패의 관점에서 수행하였다.

표 3.2: 시연 데이터 수집 매뉴얼 및 비정상 에피소드 제거 기준

구분	상세 규칙 및 제거 기준	적용 대상
기기 및 시스템 오류	녹화 중 저장 공간 부족으로 에피소드가 중단된 경우	공통
	녹화 중 카메라 연결 해제 또는 KeyboardInterrupt 발생	
	프레임 드롭 등 데이터 로깅 상의 손실이 발생한 경우	
궤적 불량 및 조작자 실수	시연 과정에서 로봇 팔의 흔들림(버벅임)이 심하게 기록된 경우	공통
	블록을 잡는 과정에서 방향을 바꾸거나 다시 고쳐 잡는 지연이 발생한 경우	
	초기 로봇팔의 위치(Home Position)가 잘못 설정된 경우 (예: 초기 손 위)	
태스크 실패 (규정 위반)	블록이 지정 구역 경계선 밖으로 과도하게 이탈한 경우 (라인 절반 이상)	Task 1, 2
	사전에 정의된 배치 또는 적재 순서가 틀린 경우	
	블록이 넓은 면으로 놓이지 않고 세워지거나 쓰러진 경우	
적재(Stacking) 특화 수집 매뉴얼	탑 붕괴 방지: 블록을 놓을 때 기존 블록에 완전히 밀착시키지 않고 약간 위에서 릴리즈(Release)하여 탑이 흔들리는 것을 방지할 것	Task 2
	충돌 회피: 무작위 배치된 블록 중 외곽에 위치한 블록부터 우선적으로 집어 로봇 팔과 다른 블록 간의 간섭을 최소화할 것	
	안전 궤적: 블록을 잡은 후 수직으로 충분히 들어 올린 뒤, 적재 위치의 상단 공간으로 이동할 것	

데이터셋을 병합하기 전에는 각 데이터셋의 Feature Schema를 비교하여 관측 상태, 행동 데이터, 영상 데이터 및 메타데이터의 구조가 서로 일치하는지를 확인하였다. 이러한 데이터 검증 과정은 단순히 전체 데이터의 양을 증가시키는 것보다 모델이 학습하기에 적합한 '최적의(Optimal)' 시연 데이터 품질과 일관성을 확보하는 데 그 목적이 있다.

3.2.8 LeRobot 데이터셋 구조 분석 및 병합·통합 관리

본 연구에서는 수집된 데이터를 LeRobot 프레임워크의 데이터셋 형식으로 관리하였다. LeRobot 데이터셋은 로봇의 상태 및 행동정보와 카메라 영상, 시간 정보, 에피소드 및 프레임 관련 메타데이터 등을 함께 관리할 수 있도록 구성되어 있어, 텔레오퍼레이션 시연 데이터를 ACT 학습에 직접 활용할 수 있도록 하였다.

각 데이터셋의 구성과 Feature Schema를 확인하여 action, observation.state, 카메라 영상, timestamp, frame_index, episode_index, index, task_index 등의 데이터를 관리하였다. 특정 Feature의 존재 여부나 자료형이 다른 경우 병합 과정에서 오류가 발생할 수 있으므로, 병합 전에 데이터셋의 메타데이터와 실제 저장 데이터를 확인하고 필요한 정제 과정을 수행하였다. 이후 데이터 수집 조건별로 별도 생성된 데이터셋을 학습 목적에 맞게 병합하여 하나의 통합 데이터셋으로 구성하였다.

이후 Multi-Task 학습을 위해 기존 데이터셋의 Feature를 확장하고 observation.environment_state와 같은 Task 조건 정보를 추가하였다. 해당 과정은 이후 3.7절에서 별도로 설명하며, 이를 통해 서로 다른 작업 데이터를 하나의 통합 데이터셋에서 관리하고 단일 정책이 Task 조건에 따라 서로 다른 작업을 수행할 수 있도록 데이터 구조를 확장하였다.

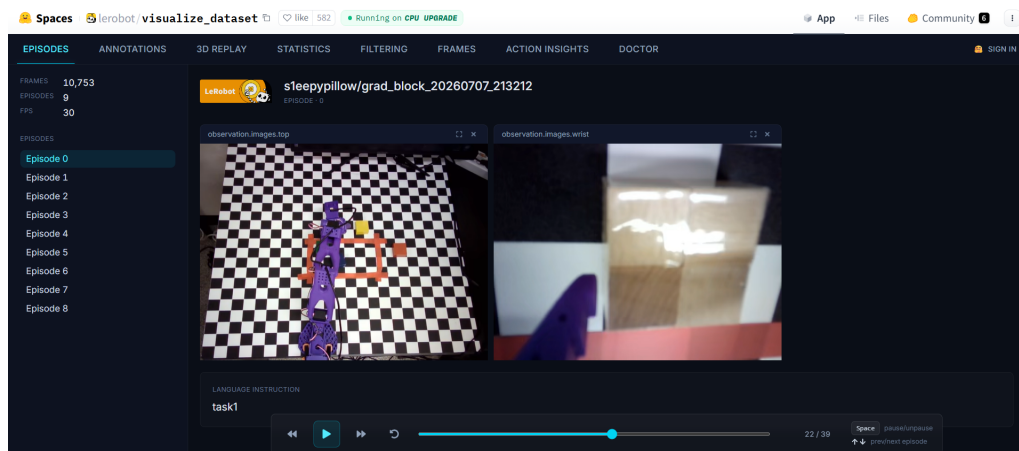


그림 3.7: Hugging Face Hub 데이터셋(Episode 목록 및 재생)



그림 3.8: Episode 관절 데이터 확인

3.3 ACT 기반 모방학습 모델 구축

3.3.1 머신러닝 학습 종류 및 선정 과정

본 연구에서는 지능형 로봇의 조작 작업을 제어하기 위해 다양한 머신러닝 및 인공지능 기법들을 검토하였으며, 최종적으로 행동 복제 기반의 모방학습을 채택하였다. 로봇 제어 분야에서 주로 활용되는 강화학습, VLA(Vision-Language-Action) 모델, World Models, Reward Models와의 비교 분석을 통해 모방학습이 가지는 타당성과 본 연구의 환경적 요구사항에 부합하는 강점을 다음과 같이 도출하였다.

기존 머신러닝 제어 기법과의 장단점 비교

각 제어 기법의 실효성을 평가하기 위해 선행 연구를 바탕으로 모델 규모, 학습 환경, 그리고 제어 주파수(Hz)를 비교 분석하였다. 상세한 정량적 수치는 표 3.3에 요약되어 있다.

- 강화학습(Reinforcement Learning, RL):** 강화학습은 자율적인 탐색을 통해 최적의 정책을 학습할 수 있다는 장점이 있으나, Pick & Stack과 같이 다단계로 이루어진 복잡한 조작 작업에서는 정교한 보상 함수 설계가 매우 어렵다는 단점이 있다. 파라미터 수 자체는 수백만(1M 10M) 단위로 적은 편이나, 물리적 시뮬레이터가 아닌 실제 로봇 환경에서 직접 수행할 경우 로봇 하드웨어나 주변 환경의 파손 위험이 크며, 유의미한 성능을 달성하기까지 수백만 번 이상의 상호작용(수일 수준의 실제 시간)이 요구되는 극심한 샘플 비효율성을 가진다.
- VLA (Vision-Language-Action) Models:** 최근 주목받는 RT-2(Brohan et al., 2023), OpenVLA(Kim et al., 2024) 등의 범용 로봇 모델은 시각적 언어 이해 능력을 결합하여 뛰어난 일반화 성능을 보여준다. 그러나 최소 50억(5B) 개에서 550억(55B) 개 이상의 파라미터를 가진 초거대 모델로, 학습을 위해 수십 대의 고성능 GPU(예: A100 64장)와 수주의 시간이 필요하다. 또한 추론 지연시간이 길어 최대 5 10Hz 수준의 동작만을 지원하므로, 30Hz 이상의 고주파수 실시간 제어가 필수적인 본 연구 환경에는 부적합하다.
- World Models:** 환경의 동역학을 모델링하여 에이전트가 내부의 '상상'을 통해 학습할 수 있도록 하는 기법(예: DreamerV3)이다. 샘플 효율성을 높일 수 있으나, 고차원의 이미지 상태 공간을 예측하기 위한 모델 구조가 복잡하여 약 1억 2억(100M 200M) 개의 파라미터를 요구하며, 학습 과정에서 높은 연산 오버헤드가 발생한다.

- **Reward Models (RLHF 등):** 인간의 피드백을 통해 보상 모델을 직접 학습하는 방식은 보상 함수 설계의 어려움을 일부 해소할 수 있으나, 선호도 데이터를 추가로 수집해야 하는 부담이 존재한다. 보상 모델 학습 후 이를 기반으로 다시 정책 모델을 훈련해야 하므로 파이프라인의 복잡도가 크게 증가한다.

표 3.3: 주요 로봇 제어 학습 모델별 정량적 지표 비교 (선행 연구 기준)

학습 방식	대표 모델	파라미터 수	학습 환경 및 소요 시간	추론 주파수	특징 및 한계
VLA Models	OpenVLA (Kim et al., 2024)	약 70억 (7B)	64×A100 GPU 약 2주	5 ~ 10 Hz	높은 범용성 단일 GPU 학습 불가
VLA Models	RT-2 (Brohan et al., 2023)	5B ~ 55B	수백 대 TPU 수주	1 ~ 5 Hz	높은 범용성 막대한 연산량 및 실시간성 부족
RL (실제 로봇)	SAC, PPO 기반 정책	1M ~ 10M	실제 환경 상호작용 수백만 회 / 수일	30+ Hz	높은 제어 주파수 극심한 샘플 비효율 및 하드웨어 파손 위험
World Models	DreamerV3 (Hafner et al., 2023)	100M ~ 200M	단일/다중 GPU 수일	15 ~ 30 Hz	환경의 잠재적 동역학 학습 복잡한 파이프라인 및 연산 오버헤드
Imitation Learning	ACT (Zhao et al., 2023)	약 8,000만 (80M)	1× RTX 3090/4090 8 ~ 10시간	50+ Hz	높은 샘플 효율 엣지 디바이스 배포 용이

모방학습의 채택 계기 및 경량화의 강점

위의 비교 분석을 바탕으로, 본 연구는 설계 복잡도를 낮추면서도 높은 제어 성능을 보장할 수 있는 **모방학습(특히 ACT 기반 모델)**을 최종 제어 알고리즘으로 채택하였다. 모방학습은 사람이 직접 시연한 고품질의 데이터를 지도학습 방식으로 모사하므로 보상 함수 설계나 위험한 무작위 탐색 과정이 불필요하다.

특히 본 연구에서 모방학습을 선택한 가장 핵심적인 이유는 표 3.3에서 확인되듯 **'모델의 경량화와 그로 인한 실시간성 확보'**에 있다.

1. **파라미터 수 및 학습 시간의 효율성:** 초거대 모델인 VLA(수십억 파라미터)와 달리, 본 연구에서 구축한 ACT 기반 모방학습 모델은 ResNet 기반의 시각적 인코더와 경량화된 트랜스포머 인코더-디코더 구조를 사용하여 전체 파라미터 수가 약 8,000만(80M) 수준에 불과하다. 이는 연구실 수준의 단일 GPU 머신(RTX 3090/4090 등)에서도 수 시간(2 4시간) 내에 수천 에피소드 분량의 모델 학습을 완료할 수 있게 하며, 하이퍼파라미터 튜닝과 실험 주기를 획기적으로 단축시키는 이점을 제공한다.
2. **로컬/서버 추론 유연성 및 실시간 제어 성능:** 지능형 로봇의 조작 작업에서는 카메라 이미지를 입력받아 관절의 목표 위치를 출력하기까지의 지연시간 최소화가 필수적이다. ACT 모델은 가벼운 연산량 덕분에 50Hz 이상의 고속 추론이 가능하다. 따라서 Jetson Orin Nano와 같은 엣지 컴퓨팅 보드에 모델을 직접 올려 수행하는 로컬 추론 환경 구축이 용이하다. 또한, 컴퓨팅 자원의 한계가 있는 엣지 디바이스의 부하를 줄이기 위해

제어 PC로 영상만을 전송하여 추론하는 서버-클라이언트 방식을 적용하더라도, 네트워크 오버헤드를 포함한 전체 제어 주기를 실시간 제어의 기준선인 30 50Hz 수준으로 원활히 유지할 수 있다.

결과적으로, ACT 기반 모방학습은 거대 모델 대비 100분의 1 수준의 파라미터만으로도 시각-행동 매핑을 훌륭히 수행해낸다. 학습 파이프라인이 직관적일 뿐만 아니라 파라미터 최적화를 통한 가벼운 연산량을 자랑하므로, 제한된 컴퓨팅 자원 내에서 즉각적이고 부드러운 로봇 암 모션 제어를 달성하기 위한 가장 합리적이고 실용적인 선택이다.

3.3.2 모방학습 모델 종류 및 선정 과정

최근 로봇 매니플레이션 분야의 모방학습에서는 인간 시연 데이터가 가지는 다중 모달리티와 비마코프 특성을 해결하기 위해 다양한 생성형 인공지능 기법들이 도입되고 있다. 본 연구에서는 효과적인 블록 수직 적재(Pick & Stack) 작업을 수행하기 위해 Diffusion Policy, VQ-BeT, Multitask DiT Policy 등 최신 모방학습 모델들의 특성을 비교 분석하였으며, 최종적으로 ACT(Action Chunking Transformer)를 선정하였다. 각 모델의 특징과 ACT 선정 계기는 다음과 같다.

기존 모방학습 모델의 특징 및 한계

- **Diffusion Policy 및 Multitask DiT Policy:**

Diffusion 모델 기반의 정책은 노이즈 제거 과정을 통해 매우 복잡하고 다형적인 행동 분포를 학습하는 데 뛰어난 성능을 보인다. 특히 DiT(Diffusion Transformer) 구조를 활용한 모델은 대규모 멀티태스크 학습에서 우수한 일반화 성능을 입증하였다. 그러나 추론 과정에서 반복적인 역확산 스텝이 요구되어 연산량이 매우 높다. 이는 본 연구에서 구축한 Jetson Orin Nano 기반의 엣지 환경이나 실시간 통신 제어 환경에서 제어 주기를 지연시키는 치명적인 단점으로 작용한다.

- **VQ-BeT (Vector Quantized Behavior Transformer):**

VQ-BeT는 연속적인 행동 공간을 이산화하여 다중 모달리티 문제를 해결하는 접근법이다. 그러나 VQ-VAE를 통한 양자화 과정에서 발생하는 오차로 인해, 정밀한 위치 제어와 미세한 그리퍼 조작이 필수적인 수직 적재 작업에서 세밀한 제어 성능이 저하될 우려가 있으며 모델 학습 파이프라인이 상대적으로 복잡하다.

ACT(Action Chunking Transformer)의 장점 및 최종 선정 계기

본 연구에서 ACT를 최종 모델로 채택한 핵심 이유는 **실시간 제어 효율성**과 **부드러운 궤적 생성 능력**의 조화에 있다. 구체적인 선정 이유는 다음과 같다.

1. Action Chunking과 Temporal Ensembling을 통한 제어 안정성:

ACT는 한 번의 추론으로 일정 길이의 행동 시퀀스를 동시에 예측한다. 이후 중첩되는 행동 청크들을 시계열적으로 앙상블하여 실행함으로써, 비전 기반 제어에서 흔히 발생하는 에러 누적을 억제한다. 이는 로봇 팔의 떨림이나 바닥과 충돌하는 현상을 최소화하여 수직 적재 작업 시 매우 부드럽고 안정적인 궤적을 보장한다.

2. CVAE 기반의 다중 모달리티 처리:

잠재 변수를 활용하는 CVAE(Conditional Variational Autoencoder) 구조를 트랜스포머 인코더-디코더와 결합하여, 인간 시연자가 보여주는 다양한 궤적과 방식(다중 모달리티)을 효과적으로 모델링할 수 있다.

3. 단일 Forward Pass를 통한 빠른 추론 속도:

Diffusion 기반 정책과 달리 한 번의 Forward Pass만으로 다수의 행동 스텝을 예측하므로 추론 지연시간이 매우 짧다. 본 연구의 TCP Socket 기반 실시간 서버-클라이언트 통신 구조에서, ACT의 빠른 추론 속도는 네트워크 지연을 상쇄하고 안정적인 제어 주기를 확보하는 데 결정적인 역할을 한다.

결론적으로, 다중 블록 적재 작업에 요구되는 정밀하고 연속적인 작업 수행 능력과 시스템 하드웨어 제약(지연시간 최소화)을 동시에 만족시키기 위해, 연산 효율성이 뛰어나면서도 높은 제어 안정성을 제공하는 ACT 모델이 본 연구 환경에 가장 적합하다고 판단하였다.

3.3.3 Action Chunking Transformer(ACT) 기반 정책 구성

본 연구에서는 로봇 제어 정책 학습을 위해 Action Chunking Transformer(ACT)를 채택하였다. ACT는 트랜스포머 인코더-디코더 구조를 기반으로 하며, 카메라 이미지와 로봇의 관절 상태(observation.state)를 입력으로 받아 향후 일정 구간(chunk)에 해당하는 연속된 액션 시퀀스를 예측하는 방식으로 동작한다. [6] 매 시점마다 단일 액션을 예측하는 방식과 달리, 여러 스텝에 걸친 액션을 한 번에 예측함으로써 예측 노이즈를 완화하고 동작의 연속성을 확보할 수 있다는 점에서 로봇 팔 제어에 적합한 구조로 판단하여 적용하였다. [7]

또한 ACT는 조건부 변분 오토인코더(Conditional Variational Autoencoder, CVAE) 구조를 내재하고 있다. 학습 시에는 인코더가 실제 시연 액션 시퀀스로부터 잠재 변수(latent

variable) z 의 분포 파라미터(평균, 로그분산)를 추정하고, 이를 디코더에 조건으로 제공하여 액션을 예측한다. 반면 추론 시에는 정답 액션이 존재하지 않으므로 인코더가 작동하지 않으며, z 는 고정값(표준정규분포의 평균)으로 대체된다. 이러한 구조는 동일한 상황에 대해 시연자마다 다르게 나타나는 동작 스타일의 편차를 잠재 변수가 흡수하도록 설계된 것으로, 본 연구에서 다수의 시연자가 나누어 데이터를 수집한 환경과 밀접한 관련이 있다(관련 분석은 3.6절 참조).

3.3.4 학습 파라미터 설정

ACT 모델의 학습은 별도의 GPU 서버에서 수행하였다. 학습 과정에서는 3.2절에서 구축한 LeRobot 데이터셋을 입력으로 사용하였으며, CUDA 기반 GPU 연산을 적용하였다.

학습에 사용한 주요 파라미터는 다음과 같다.

표 3.4: ACT 학습 주요 파라미터

항목	설정값
Policy	ACT
Batch Size	64
Training Steps	100,000
Chunk Size	60
Action Steps	60
Device	CUDA
Tracking	Weights & Biases

`chunk_size`는 한 번의 정책 호출을 통해 예측하는 미래 행동의 길이를 의미하며, `n_action_steps`는 생성된 Action Chunk 중 실제 환경에서 실행할 행동의 개수를 의미한다. LeRobot ACT에서는 `n_action_steps`가 `chunk_size`보다 클 수 없도록 구성되어 있으며, 본 연구에서는 두 값을 모두 60으로 설정하여 한 번의 정책 실행에서 생성된 60개의 행동을 순차적으로 적용하는 방식으로 구성하였다.

학습 과정에서는 Weights & Biases(W&B)를 이용하여 Loss 등의 학습 지표를 기록하였다. 이를 통해 학습 과정에서 손실 값의 변화와 학습 진행 상태를 확인하고, 학습 완료 후 모델의 학습 결과를 분석할 수 있도록 하였다.

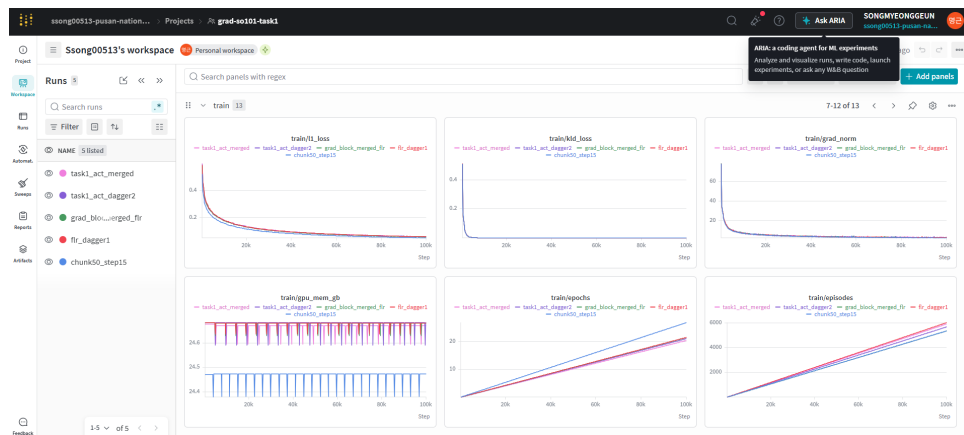


그림 3.9: ACT 학습 과정의 Loss 변화

학습 과정에서 기록된 Loss 변화를 나타낸다. 해당 결과를 통해 학습 단계에 따른 정책의 손실 변화와 학습 수렴 과정을 확인하였다.

3.3.5 학습 파라미터 변경에 따른 결과 변화 - TODO

3.3.6 Training → HF → Deployment

ACT 학습이 완료된 이후 생성된 정책 모델은 추론 단계에서 사용할 수 있도록 별도의 모델 저장소에서 관리하였다. 본 연구에서는 Hugging Face Hub를 이용하여 학습된 ACT 정책의 모델 파일과 설정 정보를 관리하였다.

학습 및 배포 과정은 크게 데이터셋 준비, ACT 학습, 학습 결과 저장, Hugging Face Hub 업로드, 추론 환경에서의 모델 로딩 단계로 구성하였다. 데이터셋은 3.2절에서 구축한 LeRobot 데이터셋을 사용하였으며, 학습 서버의 GPU 환경에서 ACT 정책을 학습하였다. 학습이 완료되면 정책의 Checkpoint 및 관련 설정 정보를 저장하고 Hugging Face Hub에 업로드하여 추론 환경에서 동일한 정책을 불러올 수 있도록 구성하였다.

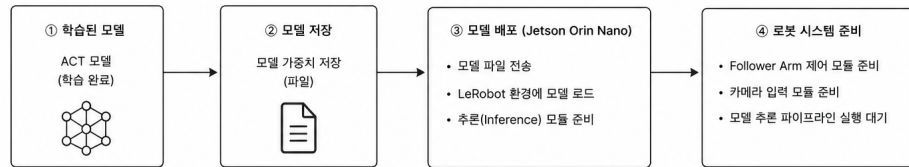


그림 3.10: ACT 모델 학습 및 배포 흐름

그림 3.10과 같이 본 연구의 정책 학습 및 배포 과정은 *Dataset* → *ACT Training* → *Checkpoint* → *Hugging Face Hub* → *Inference*의 단계로 구성하였다.

Hugging Face Hub에 저장된 정책은 이후 추론 과정에서 불러와 카메라 영상과 로봇 상태를 입력으로 사용한다. 추론 과정에서 ACT는 입력된 관측 정보를 기반으로 Action Chunk를 생성하며, 생성된 행동은 실제 SO-101 Follower 로봇의 관절 제어에 사용된다. 다만 본 연구에서는 초기 실험 단계에서 Jetson Orin Nano에서 정책을 직접 실행하는 로컬 추론 방식도 구성하여 테스트하였으며, 이후 실제 실험을 통해 서버 기반 추론 방식으로 구조를 변경하였다. 로컬 추론 방식과 서버-클라이언트 기반 분산 추론 구조의 설계 및 비교 과정은 3.4절에서 자세히 설명한다.

이를 통해 데이터 수집, 정책 학습, 모델 관리 및 실제 로봇 적용을 분리하여 관리할 수 있는 ACT 기반 모방학습 파이프라인을 구성하였다.

3.4 서버-클라이언트 기반 실시간 추론

본 연구에서는 학습된 ACT 정책을 실제 로봇에 적용하기 위한 실시간 추론 시스템을 구축하였다. 초기에는 Jetson Orin Nano에서 정책 추론을 직접 수행하는 로컬 추론 방식으로 시스템을 구성하였으나, 실제 로봇 실험 과정에서 추론 연산과 카메라 영상 처리 및 로봇 제어를 하나의 엣지 장치에서 동시에 수행하는 구조를 검토할 필요가 있었다.

또한 중간 연구 과정에서 지도교수 및 산학협력 멘토의 피드백을 통해 로컬 추론과 서버 추론 방식 중 실제 시스템에 적합한 구조를 명확하게 결정할 필요성이 제기되었다. 이에 따라 두 방식의 구조를 각각 구성하여 실제 환경에서 비교하였으며, 실시간 로봇 제어 성능을 고려하여 최종적으로 GPU 서버에서 정책 추론을 수행하고 Jetson Orin Nano에서는 카메라 및 로봇 제어를 담당하는 서버-클라이언트 기반 구조로 변경하였다.

최종 시스템에서는 Jetson Orin Nano가 로봇의 현재 관절 상태와 카메라 영상을 수집하여 추론 서버로 전달한다. 서버는 전달받은 관측 데이터를 ACT 정책에 입력하여 다음 행동을 생성하고, 생성된 행동 명령을 다시 Jetson으로 전송한다. Jetson은 전달받은 명령을 팔로워 압에 적용하여 실제 작업을 수행하도록 구성하였다.

3.4.1 서버-클라이언트 분산 추론 구조 설계

최종 시스템은 크게 Jetson Client, 추론 Server, SO-101 Follower로 구성하였다. Jetson Orin Nano는 실제 로봇과 직접 연결되어 있으므로 카메라 영상 및 로봇 상태 수집과 제어 명령 적용을 담당하도록 구성하였다. 반면 GPU 서버는 ACT 정책을 실행하여 입력된 관측 데이터로부터 로봇 행동을 생성하는 역할을 담당한다.

Jetson에서는 매 제어 단계마다 현재 로봇 상태와 카메라 영상을 수집하고, 이를 네트워크를 통해 추론 서버로 전달한다. 서버에서는 수신된 데이터를 ACT 정책에 입력하여 행동을 생성하며, 결과를 다시 Jetson으로 반환한다.

이러한 구조를 적용함으로써 로봇과 직접 연결된 Jetson에서는 센서 및 제어 관련 처리를 담당하고, 상대적으로 높은 연산량이 요구되는 정책 추론은 GPU 서버에서 수행하도록 역할을 분리하였다. 이를 통해 실제 로봇 제어 장치와 정책 추론 환경을 독립적으로 구성할 수 있도록 하였다.

3.4.2 TCP Socket 기반 통신 프로토콜 및 Jetson Client 구현

Jetson Orin Nano와 추론 서버 사이의 데이터 교환을 위해 TCP Socket 기반 통신 방식을 구현하였다. Jetson Client는 실제 로봇과 연결된 장치로서 카메라 영상과 로봇의 현재 관절 상태를 수집하고, 이를 추론 서버에 전달하는 역할을 수행한다.

카메라 영상은 네트워크를 통해 전송하기 위해 JPEG 형식으로 압축하였다. 또한 로봇의 관절 상태와 압축된 카메라 영상을 하나의 요청 데이터로 구성하여 서버로 전송하도록 구현하였다. 서버에서 추론이 완료되면 생성된 행동 명령을 다시 Jetson Client로 전달하고, Jetson에서는 전달받은 명령을 Follower 로봇팔에 적용한다.

하나의 추론 요청은 다음과 같은 순서로 처리된다.

1. Jetson에서 상단 및 손목 카메라의 최신 영상을 확보한다.
2. Follower 로봇의 현재 관절 상태를 읽는다.
3. 카메라 영상을 JPEG 형식으로 압축한다.
4. 압축된 영상과 로봇 상태를 TCP Socket을 통해 추론 서버로 전송한다.
5. 서버에서 ACT 정책의 추론 결과를 수신한다.
6. 수신한 행동 명령을 Follower 로봇의 관절 제어에 적용한다.

실시간 영상 수집을 위해 카메라별 Worker를 별도로 구성하였다. 각 Worker는 카메라에서 영상을 지속적으로 수집하며, 메인 제어 루프에서는 가장 최근에 확보된 프레임을 사용하도록 구성하였다. 이를 통해 메인 제어 루프가 카메라 프레임의 개별 수집 과정이 완료될 때까지 대기하는 상황을 줄이고자 하였다.

또한 TCP 통신 과정에서는 TCP_NODELAY 옵션을 적용하였다. 이를 통해 작은 크기의 데이터가 전송 버퍼에 대기하면서 발생할 수 있는 불필요한 지연을 줄이고, 관측 데이터와 행동 명령의 실시간 전달을 고려하였다.

이와 같이 Jetson Client는 카메라 및 로봇 상태 수집, 데이터 전송 및 수신된 행동 명령의 적용을 담당하며, 정책 추론 자체는 별도의 GPU 서버에서 수행하도록 역할을 분리하였다.

3.4.3 서버 측 ACT 정책 추론 및 카메라 영상 전송 구현

앞선 절에서 구성한 서버-클라이언트 구조를 실제 Pick & Stack 작업에 적용하였다. Jetson Orin Nano는 작업 공간의 카메라 영상과 팔로워 암의 현재 상태를 지속적으로 수집하며, 추론 서버와의 통신을 통해 정책 실행에 필요한 데이터를 전달하고 결과로 생성된 행동 명령을 로봇에 적용한다.

실제 Task 수행 과정에서는 블록의 현재 위치와 로봇의 자세가 지속적으로 변화하므로, 한 번의 추론 결과만으로 전체 작업을 수행하는 것이 아니라 현재 상태를 기준으로 다음 행동을 반복적으로 결정하도록 구성하였다. 이에 따라 블록 접근, 파지, 이동 및 적재 과정에서 변화하는 작업 환경을 새로운 관측 데이터에 반영할 수 있도록 하였다.

데이터 수집 및 모델 학습 과정에서 생성된 정책을 실제 시스템에 적용한 후, Jetson과 추론 서버 사이의 데이터 교환을 통해 팔로워 암이 Pick & Stack 작업을 수행하도록 구성하였다.

이 과정에서 Jetson Orin Nano는 카메라 및 로봇과 직접 연결된 엣지 장치로서 현장의 데이터 수집과 로봇 제어를 담당하고, GPU 서버는 정책 실행을 담당하도록 역할을 분리하였다. 이를 통해 실제 로봇 시스템의 제어 환경과 정책 실행 환경을 분리한 상태에서 학습된 모델을 적용할 수 있도록 하였다.

본 절에서 구성한 실시간 실행 과정의 각 단계별 처리 시간은 다음 절에서 측정하여 전체 제어 주기의 성능과 병목 구간을 분석하였다.

3.5 실시간 추론 및 제어 성능 최적화

3.4절에서 구축한 서버-클라이언트 기반 실시간 추론 구조를 실제 로봇 시스템에 적용한 이후, 안정적인 실시간 제어를 위해 각 처리 단계에서 발생하는 지연시간을 측정하고 병목 구간을 분석하였다. 실제 로봇 제어에서는 카메라 영상 획득, 로봇 상태 수집, 영상 압축, 네트워크 통신, 정책 추론 및 로봇 제어가 반복적으로 수행되므로 특정 단계의 처리 시간이 증가할 경우 전체 제어 주기가 느려질 수 있다.

이에 따라 본 연구에서는 Jetson Client에서 단계별 처리 시간을 측정할 수 있도록 디버깅 및 성능 측정 기능을 추가하고, 전체 처리 시간을 기준으로 실시간 제어 주기를 분석하였다. 또한 측정 결과를 바탕으로 불필요한 처리와 병목 구간을 확인하고, 카메라 처리 방식과 출력 방식 등을 조정하여 실시간성을 개선하였다.

3.5.1 구간별 지연시간 측정

실시간 추론 과정의 성능을 정량적으로 확인하기 위해 하나의 제어 루프를 여러 처리 구간으로 나누어 각각의 소요 시간을 측정하였다. 측정 대상은 카메라 프레임 획득, 로봇 상태 수집, JPEG 영상 압축, 데이터 송신, 서버로부터의 추론 결과 수신, 로봇 제어 및 전체 처리 시간으로 구성하였다.

각 단계의 처리 시간은 제어 루프 내부에서 측정하여 전체 처리 시간과 비교하였다. 또한 전체 처리 시간으로부터 초당 처리 가능한 제어 횟수를 계산하여 실제 시스템의 제어 주기를 확인하였다.

성능 측정 과정에서는 다음과 같은 항목을 기준으로 처리 시간을 기록하였다.

이와 같이 구간별 처리 시간을 분리하여 측정함으로써 단순히 전체 제어 주기만 확인하는 것이 아니라 실제 시스템에서 어느 처리 단계가 전체 지연시간에 가장 큰 영향을 미치는지 확인할 수 있도록 하였다.

표 3.5: 실시간 추론 제어 루프의 성능 측정 항목

측정 항목	내용
Camera	카메라 프레임 획득 시간
State	로봇 관절 상태 수집 시간
JPEG	카메라 영상 JPEG 압축 시간
Send	추론 서버로 데이터 송신 시간
Recv	추론 결과 수신까지 소요된 시간
Robot	수신된 명령의 로봇 적용 시간
Total	하나의 전체 제어 루프 처리 시간

3.5.2 병목 구간 분석 및 개선

측정된 처리시간을 분석한 결과, 전체 제어 주기는 단일 연산에 의해서만 결정되는 것이 아니라 카메라 처리, 영상 압축, 네트워크 통신 및 서버의 정책 추론 시간이 연속적으로 누적되는 구조임을 확인하였다. 특히 실시간 영상 데이터를 매 제어 주기마다 처리하기 때문에 카메라 프레임의 획득 및 JPEG 압축 과정에서 발생하는 지연시간을 함께 고려할 필요가 있었다.

이에 따라 Jetson Client에서는 카메라별 Worker를 별도로 구성하고, 카메라 영상을 지속적으로 갱신하도록 구현하였다. 메인 제어 루프에서는 카메라에서 새로운 프레임을 직접 획득하기 위해 대기하는 대신 Worker가 가장 최근에 확보한 프레임을 사용하도록 구성하였다. 이를 통해 카메라의 프레임 획득 과정이 전체 제어 루프를 직접 지연시키는 상황을 줄이고자 하였다.

또한 개발 과정에서 각 처리 단계의 시간을 확인하기 위해 출력하던 디버그 메시지가 반복적으로 출력되면서 실시간 처리에 불필요한 오버헤드를 발생시킬 수 있음을 고려하였다. 따라서 성능 측정에 필요한 정보만 출력하도록 조정하고, 반복적인 출력으로 인해 제어 루프가 영향을 받지 않도록 디버그 출력 구조를 정리하였다.

네트워크 통신에서는 TCP Socket의 TCP_NODELAY 옵션을 적용하여 작은 크기의 데이터가 전송 버퍼에서 불필요하게 대기하는 상황을 줄이도록 하였다. 이러한 개선을 통해 카메라 처리와 통신 과정에서 발생하는 추가적인 지연을 최소화하고, 전체 제어 주기를 안정적으로 유지하도록 구성하였다.

3.5.3 Action Chunk 및 Temporal Ensembling 기반 제어 방식 실험

ACT의 Action Chunk 기반 제어 방식에 따른 로봇 동작 특성을 확인하기 위해 `chunk_size` 와 `n_action_steps` 설정을 변경하며 실험하였다. 기존 설정은 `chunk_size=60`, `n_action_steps=60` 으로, 한 번 예측한 60개의 행동을 재관측 없이 순차적으로 실행하는 방식이었다. 이 설정에서는 로봇의 움직임이 비교적 부드러웠으나, 약 30 Hz의 제어 주기를 기준으로 약 2초 동안

새로운 관측 없이 동작하기 때문에 특정 구역에 머무르거나 팔이 목표 위치에서 벗어나는 현상이 발생하였다.

반대로 `n_action_steps`를 감소시켜 재관측 빈도를 높이는 경우 환경 변화에 빠르게 대응할 수 있었으나, Action Chunk가 전환되는 구간에서 로봇 움직임이 불연속적으로 변하는 현상이 나타났다. 실험 과정에서 `chunk_size=100`, `n_action_steps=5`인 경우 Chunk 전환에 따른 덜컥거림이 발생하였으며, `chunk_size=30`, `n_action_steps=30` 및 `chunk_size=50`, `n_action_steps=15`에서는 전체적인 움직임이 상대적으로 느려지는 현상을 확인하였다.

이러한 문제를 개선하기 위해 ACT 정책에서 제공하는 Temporal Ensembling 기능을 적용하였다. Temporal Ensembling은 `n_action_steps=1`로 설정하여 매 Step마다 새로운 관측을 기반으로 행동을 예측하면서, 시간적으로 중첩되는 여러 Action Chunk의 예측값을 가중 평균하여 최종 행동을 결정하는 방식이다. 이를 통해 재관측 빈도를 높이면서도 Chunk 전환에 따른 급격한 행동 변화를 완화하고자 하였다.

Temporal Ensembling은 기존 모델을 재학습하지 않고 추론 설정만 변경하여 적용하였다. 모델 구조에 고정된 `chunk_size=60`은 유지하고, `n_action_steps`를 60에서 1로 변경하였으며 `temporal_ensemble_coeff`를 0.01로 설정하였다.

표 3.6: Temporal Ensembling 적용 전후 추론 설정

항목	기존 설정	적용 설정
<code>chunk_size</code>	60	60
<code>n_action_steps</code>	60	1
<code>temporal_ensemble_coeff</code>	None	0.01

실제 로봇에 적용한 결과 Temporal Ensembling을 사용한 경우 Chunk 전환 과정에서 발생하던 덜컥거림이 감소하여 로봇 팔의 움직임이 보다 부드러워졌으며, 그리퍼가 작업면을 강하게 접촉하는 현상도 완화되었다. 또한 블록의 파지 및 이동 성능이 기존 설정에 비해 저하되지 않았으며, 일부 실험에서는 소폭 향상되는 결과를 확인하였다.

서버 추론 결과를 수신하는 시간도 20 ms대로 측정되어 약 30 Hz의 실시간 제어 주기를 유지하는 데 큰 문제가 없음을 확인하였다. 따라서 이후 실시간 추론에서는 `n_action_steps=1`, `temporal_ensemble_coeff=0.01` 설정을 기본값으로 적용하였다.

한편 Temporal Ensembling 적용 이후에도 랜덤 위치에서 학습 데이터의 특정 위치 방향으로 이동하는 현상과 순간적으로 이동량이 0에 가까워지며 정지하는 현상은 일부 지속되었다. 이는 Action Chunk 전환에 따른 동작 불연속성과는 별개의 문제로 판단하였으며, 이후 데이터 구성 및 Posterior Collapse 관련 분석을 통해 추가적으로 개선하였다.

3.6 ACT 모델 실패 원인 분석 및 DAgger

초기 ACT 모델을 실제 로봇에 적용하여 Pick & Stack 작업을 수행한 결과, 학습 데이터에서 의도한 동작이 항상 안정적으로 재현되지 않는 문제가 확인되었다. 이에 따라 단순히 학습 횟수나 데이터 양을 증가시키는 방식에서 벗어나, 실제 Task 수행 과정에서 나타난 실패 유형을 분석하고 모델의 학습 구조 및 데이터 구성 측면에서 원인을 확인하였다.

실험 과정에서 확인된 주요 문제는 블록을 정확하게 파지하지 못하는 현상, 특정 위치에서 반복적으로 동작이 정지하거나 잘못된 방향으로 움직이는 현상, 목표 영역을 블록으로 잘못 인식하는 현상 등이었다. 또한 동일한 모델이라도 블록의 위치와 접근 방향에 따라 수행 결과에 차이가 발생하였다.

이에 따라 본 연구에서는 초기 모델의 실패 구간을 분석하고, 모델 구조에서 사용되는 VAE의 잠재변수 학습 과정과 관련된 문제 가능성을 검토하였다. 이후 실패가 발생한 상황을 다시 텔레오퍼레이터로 보정하여 추가 데이터를 수집하는 DAgger 기반 데이터 보강 과정을 수행하였다.

3.6.1 초기 모델의 수행 실패 유형 분석

초기 ACT 모델의 Task 수행 결과를 분석하기 위해 동일한 조건에서 반복적으로 로봇을 실행하고, 각 실행 과정에서 발생하는 실패 유형을 기록하였다. 이를 통해 단순한 성공 및 실패 여부뿐만 아니라 어느 단계에서 작업이 실패하는지를 구분하였다.

실험에서 확인된 대표적인 실패 유형은 다음과 같다.

표 3.7: 초기 ACT 모델의 주요 Task 수행 실패 유형

실패 유형	관찰 내용
파지 실패	블록에 접근하였으나 그리퍼가 블록을 정확하게 잡지 못하는 현상
위치 편향	특정 위치의 블록에 대해 접근 또는 파지 동작이 불안정해지는 현상
반복 정지	특정 구간에서 로봇의 움직임이 반복적으로 정지하거나 동일한 동작을 수행하는 현상
목표 영역 오인식	블록이 아닌 목표 영역을 블록으로 판단하여 잘못된 동작을 수행하는 현상
접근 방향 편향	블록의 위치에 따라 특정 방향으로 접근하는 동작이 과도하게 나타나는 현상

이러한 실패 유형을 분석한 결과, 초기 모델은 학습 데이터에 포함된 일반적인 Pick & Stack 동작은 일부 수행할 수 있었지만, 블록의 위치가 달라지거나 특정 접근 상황이 발생했을 때 동작의 안정성이 감소하는 경향을 보였다.

특히 동일한 위치 또는 유사한 상황에서 로봇이 비슷한 동작을 반복하는 현상이 관찰되었으며, 이는 현재 관측에 따라 다양한 행동을 선택하기보다는 학습 과정에서 특정 행동 패턴으로 수렴했을 가능성을 고려하게 하였다.

또한 목표 영역과 블록을 구분하지 못하는 현상은 카메라 관측 정보만으로 작업 대상과 목표 영역의 위치 관계를 충분히 학습하지 못했을 가능성을 보여주었다. 따라서 데이터의 단순한 양적 증가뿐만 아니라 실패가 발생하는 상황을 포함한 데이터 보강이 필요하다고 판단하였다.

3.6.2 CVAE 구조 관점에서의 Posterior Collapse 추정 및 하이퍼파라미터 실험

ACT는 행동의 다양한 양상을 표현하기 위해 VAE 기반의 잠재변수 구조를 사용할 수 있으며, 학습 과정에서 시연 데이터의 행동 분포를 잠재공간에 표현한다. 본 연구에서는 초기 모델에서 관찰된 반복적인 행동 및 특정 상황에서의 동작 정지를 분석하는 과정에서 VAE의 잠재변수가 충분히 활용되지 않는 Posterior Collapse 가능성을 고려하였다.

Posterior Collapse는 VAE 학습 과정에서 인코더가 생성한 잠재변수가 입력 데이터의 정보를 충분히 반영하지 못하고, 사전분포에 가까운 형태로 수렴하면서 잠재변수의 정보 활용이 감소하는 현상을 의미한다. 이러한 현상이 발생할 경우 서로 다른 시연 상황에서 나타나는 행동의 차이가 잠재공간에 충분히 반영되지 않을 가능성이 있다.

본 연구에서는 이러한 가능성을 확인하기 위해 ACT 학습 과정에서 VAE 관련 하이퍼파라미터를 변경하면서 모델의 학습 및 실제 Task 수행 결과를 비교하였다. 특히 VAE의 잠재변수 학습에 영향을 주는 KL divergence 관련 가중치(`kl_weight`)를 기존 기본값인 10.0에서 1.0으로 조정하여, 잠재공간의 정보 활용 정도가 모델의 행동 다양성과 수행 결과에 미치는 영향을 확인하였다.

또한 W&B를 이용하여 학습 과정의 Loss 변화를 기록하고, 하이퍼파라미터 변경 전후의 학습 양상을 비교하였다. 이를 통해 단순히 Training Loss가 감소하는 것뿐만 아니라 실제 로봇의 Task 수행 결과와 함께 모델의 학습 상태를 평가하였다.

W&B 학습 로그 분석 결과, 두 조건 모두 학습 후반부의 KLD Loss가 10^{-5} 수준의 매우 낮은 수치로 수렴하는 양상을 보였다. `kl_weight`를 1.0으로 설정한 모델이 10.0 조건에 비해 상대적으로 미세하게 높은 수치를 유지하긴 하였으나, 전반적으로 단순히 `kl_weight`를 낮추는 것만으로는 잠재공간의 다변화(CVAE Latent Structure의 완전한 복원)를 완벽히 유도하기에는 한계가 있음을 시사한다.

그러나 실제 로봇을 통한 Task 수행 평가에서는 부분적인 행동 변화가 관찰되었다. `kl_weight`를 10.0으로 설정한 모델에서는 관측 이미지 변화에 수용적이지 못하고 과거 학습 데이터의 평균적 궤적으로 수렴하여 목표물 부근에서 헛손질을 하거나 특정 고정 위치로 제어 출력이

쏟리는 현상이 다소 경직되게 나타났다. 반면 `kl_weight`를 1.0으로 낮춘 모델에서는 이러한 고정 위치 쏠림 현상이 일부 완화되고, 목표물 접근 시 관측 정보에 다소 민감하게 반응하여 헛손질 빈도가 일부 감소하는 등 소폭의 제어 양상 개선을 보였다.

다만 실험 결과만으로 Posterior Collapse가 모델 실패의 단일 원인이라고 확정하기는 어려웠으며, 데이터의 위치 편향, 카메라 관측 특성, 파지 동작의 난이도 및 로봇 캘리브레이션 등의 다른 요인도 함께 영향을 미칠 수 있다고 판단하였다. 따라서 이후에는 모델의 하이퍼파라미터 조정과 함께 실제 실패 상황을 직접 포함하는 데이터 보강을 수행하였다.

3.6.3 기존 행동 복제의 한계와 여러 대안의 검토, 그리고 DAgger의 선택

기존의 행동 복제 방식은 전문가의 시연 궤적을 지도 학습 형태로 모델에 직접 모사시키는 방법으로, 빠르고 직관적인 학습이 가능하다는 장점이 있다. 그러나 행동 복제 모델은 학습 데이터의 분포 내에서는 우수한 성능을 보이지만, 실제 환경에서 미세한 예측 오차가 발생할 경우 로봇이 경험해 보지 못한 새로운 상태(Out-of-Distribution, OOD)로 진입하게 되며, 이러한 오차가 누적되어 최종적으로 작업 실패로 이어지는 문제를 내포하고 있다. 앞서 확인한 하이퍼파라미터 튜닝만으로는 이러한 OOD 상황에서의 물리적 대응력 부재를 근본적으로 해결하기 어려웠다.

이러한 문제를 극복하기 위해 본 연구에서는 다음의 세 가지 주요 대안적 방법론을 우선적으로 검토하였다.

- **데이터 증강:** 이미지에 노이즈를 주입하거나 크롭 등을 적용하여 시각적 강건성을 높일 수 있으나, 이미 오차가 발생해 궤적을 이탈한 상황에서 로봇이 제자리로 돌아오는 물리적인 '복구 행동'을 직접적으로 가르쳐주지는 못한다는 한계가 있다.
- **도메인 무작위화:** 주로 시뮬레이션 환경에서 다양한 물리 매개변수를 무작위화하여 현실 환경의 불확실성에 대비하는 기법이지만, 실물 하드웨어 기반으로만 시연 및 검증이 이루어지는 본 연구 환경에는 즉각적으로 적용하기에 구조적 제약이 따랐다.
- **강화학습 기반 미세조정:** 모델이 환경과 상호작용하며 스스로 최적의 정책을 찾는 강력한 방법이다. 그러나 본 연구의 Pick & Stack과 같은 복잡한 다관절 로봇 조작 작업에서는 성공 시에만 보상이 주어지는 '희소 보상' 문제로 인해 샘플 효율성이 크게 떨어진다. 무엇보다 무작위 탐색 과정에서 매니퓰레이터가 바닥이나 주변 기물과 충돌하여 심각한 하드웨어 파손을 초래할 위험이 매우 크다는 치명적인 단점이 존재한다.

결과적으로 안전하고 효율적으로 OOD 문제를 해결하기 위해서는, 오차가 발생한 상황에서 로봇이 어떻게 복구해야 하는지를 가장 직관적이고 명시적으로 알려주는 방식이 필요했다.

이에 따라 본 연구에서는 전문가가 직접 실패 구간을 교정하여 고품질의 복구 궤적을 제공하는 **DAgger(Dataset Aggregation)** 알고리즘을 최적의 대안으로 채택하였다.

DAgger는 로봇이 실제 환경과 상호작용하며 생성한 궤적에 대해 전문가가 올바른 행동을 레이블링하고 이를 누적하여 모델을 재학습함으로써 상태 분포의 불일치를 근본적으로 해소한다. 특히 본 연구에서는 매니플레이터의 물리적 실시간 제어 환경에 적합한 HG-DAgger(Human-Gated DAgger)[8]의 방법론을 차용하였다.

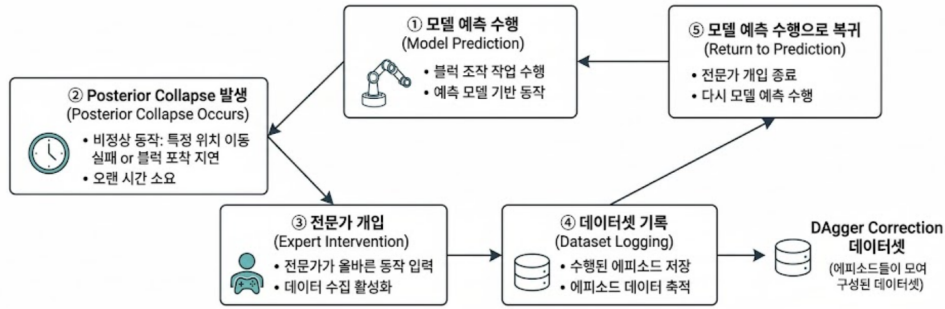


그림 3.11: DAgger Correction의 미시적 작동 흐름: 예측 수행 중 Posterior Collapse 및 OOD로 인한 실패 발생 시 전문가가 개입하여 에피소드를 수집하는 과정

그림 3.11와 같이 HG-DAgger는 로봇이 자율적으로 동작을 수행하다가 불확실성이 높아지거나 실패가 발생한 특정 구간(Sparse)에서만 인간 전문가가 텔레오퍼레이션을 통해 제어권을 획득하여 개입하고, 그 올바른 복구 궤적을 데이터로 축적하는 방식이다. 이는 데이터 수집의 효율성(로봇이 헛갈려 하는 구간만 집중 수집)을 극대화할 뿐만 아니라, 위험 상황 직전에 전문가가 개입하므로 물리적 하드웨어의 안전성을 완벽하게 보장한다는 점에서 본 연구의 방향성과 가장 잘 부합하였다.

3.6.4 시스템 파이프라인에 DAgger가 적용되는 위치와 방법

본 연구에서는 앞서 설명한 DAgger 수집 방식을 전체 모델 학습 주기에 통합하였다. 그림 3.12는 기존 ACT 학습 파이프라인에 DAgger 루프가 결합된 거시적 시스템 파이프라인을 보여준다.

통합된 파이프라인은 초기 학습, 자율 수행, 인간 개입 및 보정, 데이터 통합의 4단계 순환 구조를 가진다.

- **초기 학습(Initial Training):** 초기 시연 데이터(Base Dataset)만을 이용하여 ACT 모델을 1차적으로 학습한다.
- **자율 수행 및 평가(Autonomous Execution):** 학습된 초기 모델을 실제 로봇에 배포하여 Pick & Stack 작업을 자율적으로 수행하며 실패가 발생하는 취약 구간을 모니터링한다.

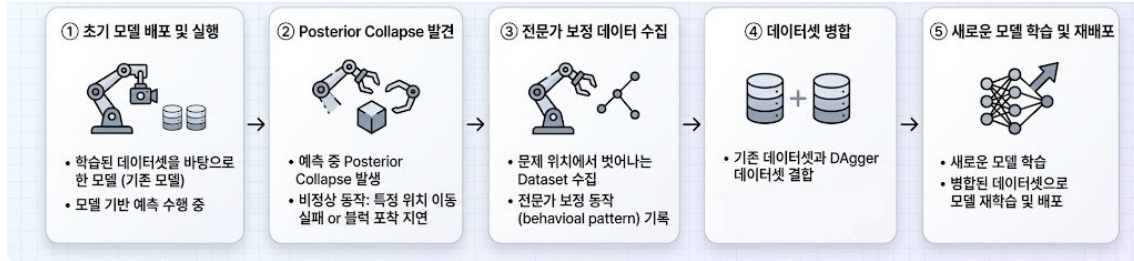


그림 3.12: 통합 시스템 파이프라인: 수집된 DAgger Correction 데이터셋이 기존 데이터셋과 병합되어 새로운 모델 학습으로 이어지는 거시적 흐름

- **인간 개입 및 보정 데이터 수집(Human Intervention & Correction):** 목표 영역 오인식, 반복 정지, 파지 실패 등의 상황이 발생할 경우, 텔레오퍼레이터가 즉시 제어권을 넘겨받는다. 전문가가 올바른 궤적으로 작업을 완수하며 생성한 실시간 상태-행동 쌍(State-Action pairs)을 문제 위치에서 벗어나는 보정 데이터로 기록한다.
- **데이터 통합 및 재학습(Aggregation & Retraining):** 수집된 DAgger Correction 데이터셋은 기존의 학습 데이터셋에 누적 병합(Aggregation)되며, 확장된 전체 데이터셋을 바탕으로 ACT 모델을 새롭게 재학습한다.

이러한 피드백 루프 구조가 결합됨으로써, 모델은 단순 시연 궤적뿐만 아니라 자신의 오차를 스스로 회복하는 능력(Recovery Behavior)까지 학습하게 되어 작업 성공률과 환경 변화에 대한 적응력이 대폭 향상된다.

3.6.5 DAgger 기반 실패 구간 보정 데이터 수집 및 반영 효과 검증

앞서 구축한 통합 시스템 파이프라인을 바탕으로, 초기 모델에서 반복적으로 나타난 블록 접근 및 파지 실패, 특정 위치에서의 동작 편향(표 3.7 참조)을 중심으로 보정 데이터를 집중적으로 확보하였다. 기존의 정상 시연 데이터를 단순히 추가하는 방식을 벗어나, 로봇이 실제 수행 과정에서 잘못된 행동을 선택한 상황을 유도하고 텔레오퍼레이션으로 궤적을 복구하는 방식으로 데이터를 구성하였다.

특히, 이러한 DAgger 기반의 복구 데이터는 ACT 모델의 CVAE 구조 내에서 발생하는 Posterior Collapse 현상을 완화하는 데 수학적, 구조적으로 중요한 기여를 한다. 기존의 일반적인 전문가 시연 데이터는 특정 관측 상태에 대해 매끄럽고 예측 가능한 단일 행동 궤적을 주로 포함한다. 이 경우 디코더는 잠재변수(z)에 의존하지 않고 관측 상태 만으로도 행동을 쉽게 재구성할 수 있어, 학습 과정에서 인코더의 사후분포가 사전분포 $p(z)$ 로 수렴해버리는 Posterior Collapse가 발생하기 쉽다.

반면, DAgger를 통해 수집된 복구 데이터는 오류 상태라는 특정 관측 조건 아래에서 '정상

궤적으로의 급격한 복귀 행동'이라는 새로운 다중 모달(Multi-modal) 행동 양상을 데이터셋에 주입한다. 즉, 주어진 상태에 대한 행동의 조건부 엔트로피(Conditional Entropy)를 증가시킨다. 이로 인해 디코더는 관측된 이미지 정보만으로는 현재 로봇이 '일반적인 접근 동작'을 유지해야 하는지, 혹은 '급격한 방향 전환(복귀) 동작'을 수행해야 하는지 명확히 구분할 수 없게 된다. 결과적으로 모델은 재구성 손실(Reconstruction Loss)을 최소화하기 위해 필연적으로 인코더가 생성한 잠재변수 z 에 담긴 세부 행동 궤적 정보를 적극적으로 활용해야만 한다.

따라서 DAgger 데이터의 통합은 단순한 `kl_weight` 조절을 넘어, 데이터 분포 차원에서 모델이 잠재공간(Latent Space)에 실제 궤적 교정에 필요한 의미 있는 행동 특징을 강제적으로 인코딩하도록 유도하여 Posterior Collapse를 근본적으로 억제하는 효과를 낳는다.

이후 기존 데이터와 수집된 보정 데이터를 통합하여 ACT 모델을 재학습하고, 동일한 조건에서 실제 로봇의 Task 수행 결과를 비교 평가하였다.

DAgger 기반 데이터 보강 이후에는 기존 모델에서 반복적으로 나타났던 일부 오류가 감소하는 것을 확인하였다. 특히 특정 상황에서 잘못된 대상을 선택하거나 블록 접근 과정에서 발생하던 일부 오류를 보정할 수 있었으며, 실패가 발생했던 상황을 직접 학습 데이터에 포함함으로써 기존 데이터만을 이용한 학습과 다른 수행 양상을 확인할 수 있었다.

그러나 DAgger 데이터 추가 이후에도 모든 실패가 해결된 것은 아니었으며, 특정 위치에서의 반복적인 동작이나 파지 실패 등 일부 문제가 계속해서 나타났다. 따라서 본 연구에서는 DAgger를 단순한 데이터 양 증가 방법으로 사용하는 것이 아니라, 초기 정책에서 발생한 실패 상황을 새로운 학습 데이터로 변환하여 데이터 분포의 부족한 부분을 보완하는 과정으로 활용하였다.

이를 통해 초기 모델의 일반적인 동작을 유지하면서 실제 Task 수행 과정에서 확인된 실패 상황에 대한 시연 데이터를 추가하였으며, 이후 통합 데이터셋 및 Task 조건화를 위한 데이터 구성으로 확장하였다.

3.7 Task 조건화를 위한 통합 데이터셋 및 Environment State

3.7.1 단일 정책을 이용한 Task 1 · Task 2 수행

본 연구의 최종 목표는 서로 다른 두 개의 Task를 하나의 신경망 모델을 이용하여 수행하는 것이었다. 이에 따라 각 Task마다 별도의 모델을 사용하는 방식이 아니라, Task 1과 Task 2의 시연 데이터를 하나의 통합 데이터셋으로 구성하고 하나의 정책이 현재 수행해야 하는 Task를 구분할 수 있도록 하는 방법을 적용하였다.

서로 다른 Task의 데이터를 단순히 하나의 데이터셋으로 병합할 경우, 카메라 영상과 로봇

상태만으로는 현재 수행해야 하는 작업의 종류를 명확하게 구분하기 어려울 수 있다. 특히 서로 다른 Task에서 유사한 로봇 상태나 시각적 관측이 나타날 수 있기 때문에, 정책이 동일한 관측에 대해 Task에 따라 서로 다른 행동을 선택할 수 있도록 별도의 Task 조건 정보가 필요하였다.

이에 본 연구에서는 LeRobot 데이터셋의 Feature Schema에 `observation.environment_state` Feature를 추가하였다. 해당 Feature는 현재 데이터가 어떤 Task에 해당하는지를 나타내는 조건 정보로 사용하였으며, Task 1과 Task 2를 서로 다른 값으로 구분하였다.

3.7.2 Environment State를 이용한 Task 조건화 설계

Task 조건 정보는 데이터셋 내부에서 각 시연 데이터가 어느 Task에 해당하는지를 나타내도록 구성하였다. 본 연구에서는 Task 1과 Task 2에 각각 고유한 Environment State 값을 할당하여 데이터를 구분하였다.

표 3.8: Task별 Environment State 구성

Task	Environment State	의미
Task 1	0	Task 1 수행 데이터
Task 2	1	Task 2 수행 데이터

여기서 `environment_state`는 로봇의 관절 상태나 작업 공간의 물리적인 환경 상태를 나타내는 값이 아니라, 현재 정책이 수행해야 하는 Task의 종류를 구분하기 위한 조건 정보로 사용하였다.

따라서 통합 데이터셋의 각 데이터는 기존의 카메라 영상 및 로봇 상태 정보와 함께 해당 데이터가 Task 1에서 수집된 것인지 Task 2에서 수집된 것인지를 나타내는 Environment State를 갖도록 구성하였다.

이러한 Task 조건을 추가함으로써 하나의 정책을 학습하면서도 서로 다른 Task의 행동 패턴을 구분할 수 있는 데이터 구조를 구성하였다.

3.7.3 통합 ACT 모델 학습 및 Task별 추론

구성된 통합 데이터셋을 이용하여 Task 1과 Task 2의 데이터를 하나의 ACT 정책 학습에 사용하였다. 학습 이후 실제 추론에서는 수행할 Task에 따라 Environment State 값을 지정하였다.

Task 1 수행 시에는 `environment_state=0`, Task 2 수행 시에는 `environment_state=1`을 사용하여 동일한 모델에 Task 조건을 전달하였다. 이를 통해 Task별로 별도의 모델을 실행하지 않고 하나의 ACT 정책에서 Task 조건에 따라 서로 다른 작업을 수행하도록 구성하였다.

최종적으로 Task 1과 Task 2의 데이터를 통합하고 Environment State를 추가함으로써 두 Task를 하나의 ACT 모델로 학습하고 Task 조건에 따라 추론할 수 있는 통합 구조를 구축하였다.

3.8 로봇 제어 안전 로직

3.8.1 정책 출력 직접 제어의 위험성 및 안전 로직 필요성

실제 로봇에 학습된 ACT 정책을 적용하는 과정에서는 정책이 생성한 행동 명령이 예상 하지 못한 관절 움직임을 발생시킬 가능성이 있다. 특히 학습 데이터에 충분히 포함되지 않은 상황에서는 특정 관절의 값이 급격하게 변하거나 로봇의 허용 범위를 벗어나는 명령이 생성될 수 있으므로, 정책 출력과 실제 로봇 제어 사이에 안전 제한을 적용하였다.

본 연구에서는 각 관절의 허용 위치 범위를 설정하고, 하나의 제어 주기에서 변경될 수 있는 최대 관절 변화량을 제한하였다. 또한 LeRobot의 로봇 제어 과정에서 제공되는 관절별 상대 변화량 제한 기능을 함께 사용하여 정책 출력이 실제 로봇에 적용되는 과정에서 과도한 움직임이 발생하지 않도록 구성하였다.

3.8.2 관절별 안전 범위 및 Step당 최대 변화량 제한 구현

SO-101 Follower의 각 관절에 대해 허용 가능한 위치 범위를 설정하고, 정책에서 생성된 제어값이 해당 범위를 벗어나는 경우 허용 범위 내로 제한하였다. 또한 이전 제어값과 현재 정책 출력값의 차이를 계산하여 한 번의 제어 주기에서 변경될 수 있는 최대 변화량을 제한하였다.

설정한 주요 안전 제한값은 다음과 같다.

표 3.9: SO-101 Follower 관절별 안전 제한값

관절	허용 위치 범위	Step당 최대 변화량
shoulder_pan	-91.0 ~ 87.0	15.0
shoulder_lift	-93.0 ~ 82.0	15.0
elbow_flex	-79.0 ~ 79.5	15.0
wrist_flex	-11.0 ~ 81.0	15.0
wrist_roll	-82.0 ~ 63.5	15.0
gripper	0.0 ~ 95.0	20.0

관절 위치 제한은 정책 출력값 자체가 설정된 범위를 벗어나지 않도록 하는 역할을 하며, Step당 변화량 제한은 연속적인 제어 과정에서 인접한 두 명령 사이의 차이가 지나치게 커지는 것을 방지한다. 특히 그리퍼는 다른 관절과 동작 특성이 다르므로 최대 변화량을 별도로 설정하였다.

3.8.3 LeRobot 프레임워크 내장 안전 기능과의 병행 적용

추가로 직접 구현한 안전 제한만 사용하는 것이 아니라 LeRobot에서 제공하는 관절별 상대 목표 변화량 제한 기능을 함께 적용하였다. 정책 출력에 대한 상위 수준의 안전 제한과 로봇 제어 단계에서의 변화량 제한을 동시에 적용하는 구조로 구성하였다.

정책 출력이 설정된 관절 범위를 벗어나거나 한 번의 제어 주기에서 허용된 변화량보다 크게 변경되는 경우에는 제한된 값이 실제 로봇 제어에 사용되도록 처리하였다. 이와 같은 이중 제한 구조를 적용하여 모델의 예측 오류나 순간적인 출력 변화가 실제 로봇의 급격한 움직임으로 직접 연결되는 것을 방지하였다.

제4장 연구 결과 분석 및 평가

4.1 데이터셋 구축 결과

본 연구에서는 SO-101 Leader-Follower 기반 Teleoperation을 통해 Pick & Stack 작업의 시연 데이터를 수집하고, ACT 모델 학습에 활용하기 위한 데이터셋을 구축하였다. 데이터셋은 블록의 위치 조건을 다양하게 구성하여 수집하였으며, 고정 위치뿐만 아니라 좌측, 우측 및 랜덤 위치에서의 작업 데이터를 포함하였다.

각 에피소드에는 상단 카메라와 손목 카메라에서 수집한 영상 정보와 팔로워 암의 관절 상태 및 Teleoperation 과정에서 생성된 행동 데이터가 함께 기록되었다. 이를 통해 블록의 시각적 위치와 로봇의 현재 상태를 행동 데이터와 함께 학습할 수 있도록 구성하였다. 수집된 데이터에 대해서는 비정상적인 동작이 포함된 에피소드를 확인하고, 데이터의 Feature Schema와 자료형 및 배열 구조를 검증하였다. 검증 과정에서 학습에 적합하지 않은 데이터를 제거하여 데이터셋의 일관성을 확보하였다.

또한 Task 1과 Task 2의 데이터를 하나의 통합 데이터셋으로 구성하고, 각 데이터에 `observation.environment.state` Feature를 추가하여 Task 정보를 구분할 수 있도록 하였다. 이를 통해 서로 다른 Task의 시연 데이터를 동일한 데이터 구조에서 관리하고 하나의 ACT 정책 학습에 활용할 수 있는 형태로 구성하였다.

최종적으로 구축된 데이터셋은 다양한 블록 위치 조건과 두 개의 Task에 대한 시연 데이터를 포함하며, 이후 ACT 모델 학습 및 실제 로봇을 이용한 Task 수행 평가에 사용하였다.

4.2 ACT 모델 학습 및 성능 평가

본 연구에서는 구축된 Teleoperation 데이터셋을 이용하여 ACT 모델을 학습하고, 실제 SO-101 Follower를 대상으로 Pick & Stack 작업 수행 성능을 평가하였다. 모델의 평가는 학습 과정에서의 Loss 변화와 실제 로봇의 Task 수행 결과를 함께 비교하는 방식으로 진행하였다.

4.2.1 초기 ACT 모델 수행 결과

초기 ACT 모델을 학습한 후 실제 로봇에 적용하여 Task 수행 결과를 확인하였다. 학습 과정에서는 Training Loss가 감소하는 경향을 확인하였으나, 실제 로봇 평가에서는 블록의 위치가 변경되는 상황에서 안정적인 작업 수행이 이루어지지 않는 경우가 발생하였다.

특히 블록을 집는 과정에서의 파지 실패와 목표 영역을 잘못 판단하는 현상이 나타났으며, 특정 상황에서는 유사한 동작을 반복하거나 작업 진행이 중단되는 현상도 관찰되었다. 이를 통해 학습 Loss의 감소만으로 실제 로봇의 Task 수행 성능을 판단하기에는 한계가 있음을 확인하였다.

초기 모델의 수행 결과를 분석한 결과, 학습 데이터에 포함된 작업 조건과 실제 평가 환경의 차이가 모델의 수행 결과에 영향을 미치는 것으로 판단하였다. 이에 따라 실제 수행 과정에서 발생한 실패 상황을 추가 데이터로 보완하여 모델을 재학습하였다.

4.2.2 DAgger 적용에 따른 성능 변화

초기 ACT 모델의 수행 과정에서 발생한 실패 상황을 보완하기 위해 DAgger 방식으로 추가적인 보정 데이터를 수집하였다. 추가 데이터는 초기 모델이 정상적으로 수행하지 못한 상황을 중심으로 Teleoperation을 이용하여 수집하고 기존 학습 데이터에 반영하였다.

DAgger 적용 전후의 실제 로봇 수행 결과를 비교한 결과, 일부 평가 조건에서 평균 수행 블록 수가 증가하였다. Fixed 조건에서는 평균 수행 블록 수가 1.4개에서 3.2개로 증가하였으며, Random 조건에서는 1.0개에서 2.2개로 증가하였다.

또한 추가 데이터를 반영한 이후 일부 실패 유형이 감소하는 것을 확인하였다. 특히 목표 영역을 블록으로 잘못 판단하는 현상이 감소하여 특정 상황에 대한 모델의 대응이 개선되는 결과를 확인하였다.

표 4.1: DAgger 적용 전후 Task 수행 결과		
평가 조건	DAgger 적용 전	DAgger 적용 후
Fixed	1.4	3.2
Random	1.0	2.2

표에서 확인할 수 있듯이 DAgger를 통한 추가 데이터 반영 이후 두 평가 조건에서 평균 수행 블록 수가 증가하였다. 이는 실제 모델이 실패한 상황을 중심으로 데이터를 보강하는 방법이 모델의 일부 수행 성능을 개선하는 데 효과가 있었음을 보여준다.

4.2.3 하이퍼파라미터 실험 결과

초기 모델에서 관찰된 반복적인 동작과 작업 중단 현상의 원인을 분석하기 위해 ACT 학습 과정에서 일부 하이퍼파라미터를 변경하여 추가 실험을 수행하였다. 각 실험에서는 동일한 평가 조건을 유지하고 학습 Loss와 실제 로봇 수행 결과를 비교하였다.

실험 결과 하이퍼파라미터 설정에 따라 반복적인 동작이나 작업 수행 결과에 차이가 나타났으나, 특정 설정만으로 모든 실패 현상이 해결되지는 않았다. 따라서 모델의 수행 성능은 학습 설정뿐만 아니라 데이터 구성과 실제 작업 환경 등 여러 요인의 영향을 받는 것으로 확인하였다.

4.3 통합 ACT 모델의 Task 수행 평가

본 연구에서는 Task 1과 Task 2의 데이터를 통합하여 학습한 하나의 ACT 모델을 실제 SO-101 Follower에 적용하고 Task별 수행 결과를 평가하였다. 평가 과정에서는 수행할 Task에 따라 `observation.environment_state` 값을 변경하여 동일한 정책에서 각 Task를 수행하도록 구성하였다.

4.3.1 Task 1 수행 결과

통합 ACT 모델의 Task 1 수행 성능을 확인하기 위해 실제 로봇을 이용하여 반복 평가를 수행하였다. 평가 과정에서는 Task 1에 해당하는 Environment State를 입력하고 블록의 파지, 이동 및 적재 과정을 중심으로 수행 결과를 확인하였다.

모델 마다 총 10회의 평가를 수행한 결과 10회의 작업에서 목표 동작을 수행하였으며, 평균 3.2개의 블록을 처리하였다. 평가 과정에서는 블록의 위치에 따라 파지 성공 여부에 차이가 나타났으며, 일부 상황에서는 블록 접근 및 파지 과정에서의 실패가 관찰되었다.

4.3.2 Task 2 수행 결과

Task 2에 대해서도 동일한 통합 ACT 모델을 사용하여 실제 로봇 수행 평가를 진행하였다. 추론 과정에서는 Task 2에 해당하는 Environment State를 입력하여 Task 조건을 지정하였다.

모델 마다 총 10회의 평가를 수행한 결과 8회의 작업에서 목표 동작을 수행하였으며, 평균 2개의 블록을 처리하였다. Task 2에서도 블록의 위치와 로봇의 초기 자세 등에 따라 수행 결과에 차이가 나타나는 것을 확인하였다.

4.3.3 Environment State 기반 Task 조건화 평가

Task 1과 Task 2의 수행 결과를 비교하여 하나의 ACT 모델에서 Environment State를 이용한 Task 조건화가 정상적으로 이루어지는지 확인하였다.

Task 1 수행 시에는 `environment_state=0`, Task 2 수행 시에는 `environment_state=1`을 사용하였다. 동일한 모델에서 입력되는 Task 조건을 변경한 결과 각각의 Task에 해당하는 정책 출력을 생성하고 실제 로봇에 적용할 수 있음을 확인하였다.

이를 통해 Task별로 별도의 ACT 모델을 사용하는 대신 하나의 통합 모델에서 Environment State를 이용하여 수행할 Task를 구분하는 방식이 실제 로봇 환경에서도 적용 가능함을 확인하였다.

4.4 실시간 추론 시스템 성능 평가

본 연구에서는 최종적으로 구축한 서버-클라이언트 기반 실시간 추론 시스템의 처리 성능을 평가하기 위해 전체 제어 루프에 소요되는 시간을 측정하였다. 제어 루프는 카메라 영상 및 로봇 상태 수집, 추론 서버와의 데이터 통신, ACT 정책 추론 결과 수신 및 로봇 명령 적용 과정을 포함한다.

4.4.1 로컬 추론과 서버 추론 비교

ACT 모델의 추론 환경에 따른 실시간 제어 성능을 비교하기 위해 Jetson Orin Nano에서 직접 추론하는 방식과 GPU 서버에서 추론하는 방식을 각각 테스트하였다.

로컬 추론 방식에서는 카메라 영상 처리와 로봇 제어뿐만 아니라 ACT 모델의 추론까지 Jetson Orin Nano에서 수행하였다. 반면 서버 기반 추론 방식에서는 Jetson Orin Nano가 카메라 및 Follower 로봇의 데이터를 처리하고 GPU 서버가 ACT 정책 추론을 수행하도록 역할을 분리하였다.

실험 결과 서버 기반 추론 방식에서 GPU의 연산 자원을 활용하여 ACT 모델의 추론을 수행할 수 있었으며, Jetson Orin Nano의 연산 부담을 줄이면서 실시간 로봇 제어가 가능한 구조를 구성할 수 있었다. 이에 따라 최종 시스템에서는 Jetson Orin Nano를 카메라 및 로봇 제어에 사용하고 GPU 서버를 정책 추론에 사용하는 구조를 적용하였다.

4.4.2 실시간 제어 지연시간 분석

최종 시스템의 실시간 처리 성능을 확인하기 위해 카메라 데이터 수집부터 ACT 추론 결과를 실제 로봇에 적용하는 과정까지 포함한 전체 제어 루프의 처리 시간을 측정하였다.

측정 결과 하나의 전체 제어 루프를 처리하는 데 평균 약 30 ms가 소요되었다. 이를 제어 주기로 환산하면 약 33.3 Hz에 해당하며, 실제 로봇 제어 과정에서 지속적으로 새로운 관측 데이터를 수집하고 정책 결과를 적용할 수 있음을 확인하였다.

표 4.2: 실시간 제어 시스템의 전체 처리 성능

평가 항목	측정 결과
전체 제어 루프 시간	약 30 ms
제어 주기	약 33.3 Hz

또한 실시간 처리 과정에서 발생하는 불필요한 디버그 출력과 카메라 프레임 처리에 따른 지연을 줄이기 위해 카메라 Worker 구조와 최신 프레임 기반 처리 방식을 적용하였다. 이를 통해 불필요하게 오래된 프레임을 처리하는 것을 방지하고 실시간 제어에 필요한 최신 관측 데이터를 우선적으로 사용할 수 있도록 구성하였다.

최종적으로 평균 약 30 ms의 제어 루프 처리 시간을 확인함으로써 구축한 서버-클라이언트 기반 추론 시스템이 실제 SO-101 Follower 로봇의 실시간 정책 실행에 활용될 수 있음을 확인하였다.

제5장 결론 및 향후 연구 방향

5.1 연구 결론 요약

본 연구에서는 무작위로 배치된 블록을 인식하고 지정된 영역에 순차적으로 적재하는 Pick & Stack 태스크를 해결하기 위해, 다수의 텔레오퍼레이션 데이터셋과 지도학습 기반의 ACT 모델을 활용한 로봇 매니플레이션 시스템을 성공적으로 구축하였다. 수집된 전문가의 시연 데이터를 바탕으로 로봇의 행동 궤적을 학습시켰으며, 이를 통해 기본적인 블록 파지 및 이동 동작을 구현할 수 있었다.

나아가, 실제 추론 과정에서 로봇이 특정 위치나 상태에 갇혀 다음 동작으로 매끄럽게 넘어가지 못하는 현상을 해결하기 위해 DAgger(Dataset Aggregation) 알고리즘을 도입하였다. 모델이 오작동을 일으키는 취약한 상태에서 전문가가 직접 개입하여 올바른 궤적으로 교정하는 데이터를 추가 수집 및 재학습함으로써, 모델의 유연성을 크게 향상시키고 상태 전환이 한층 원활하게 이루어지도록 개선하였다. 결론적으로 ACT 모델과 DAgger의 결합은 동적이고 불확실성이 존재하는 환경에서 로봇 매니플레이션의 성공률과 강건성을 확보하는데 효과적임을 입증하였다.

5.2 연구의 한계점

5.2.1 데이터셋 수집에서의 한계

본 연구의 데이터 수집 과정에서는 다수의 시연자가 참여하여 텔레오퍼레이션을 수행하였다. 이로 인해 데이터셋 자체의 일관성이 저하되었으며, 동일한 상황에서도 시연자마다 블록에 접근하는 궤적이나 그리퍼 조작 타이밍 등 행동 패턴이 다르게 기록되는 문제가 발생하였다. 이러한 시연 데이터의 높은 분산은 모델이 최적의 정책으로 수렴하는 것을 방해하는 요소로 작용하였다.

또한, 장기간에 걸친 데이터 수집 과정에서 로봇팔의 기구학적 미세 조정이나 카메라의 위치 및 각도가 미세하게 변경되는 환경적 변화가 존재하였다. 이러한 하드웨어 및 환경의 변경은 입력 데이터의 분포를 변화시키는 도메인 시프트를 유발하여, 학습의 안정성을 떨어

뜨리는 방해 요소로 작용했을 가능성을 배제할 수 없다.

5.2.2 학습 과정에서의 한계

학습 과정에서 적용된 ACT 모델과 DAgger 알고리즘은 모방학습 및 지도학습 방식을 기반으로 하므로 방법론적 한계를 내포하고 있다. 가장 큰 한계는 모델의 최종 성능이 데이터를 제공한 시연자의 조작 능력을 결코 뛰어넘을 수 없다는 점이다. 즉, 전문가의 퍼포먼스가 모델 성능의 최대치로 작용하게 된다.

더불어, DAgger를 활용한 성능 개선 과정은 물리적인 시간과 노동력이 막대하게 소모된다는 치명적인 단점이 드러났다. 전문가가 실제 로봇 제어 현장에 상주하며 로봇이 특정 상태에 갇힐 때까지 기다린 후, 수동으로 개입하여 경로를 수정하고 이를 다시 학습 데이터로 변환하는 일련의 과정은 병목 현상을 유발하며 확장성 측면에서 뚜렷한 한계를 보였다.

5.3 향후 연구 방향

5.3.1 강화학습 기반 태스크 최적화

본 연구에서는 시스템의 빠른 구축과 효율적인 유지보수를 통한 점진적인 성능 향상을 목표로 하였기 때문에, 복잡한 보상 함수 설계 및 학습 안정화가 요구되는 강화학습 대신 ACT와 DAgger를 결합한 모방학습 기반 모델을 우선적으로 채택하였다. 하지만 이러한 지도학습 기반의 접근 방식은 정답에 해당하는 전문가의 조작 수준 이상을 넘어서기 어렵다는 문제가 존재한다.

이러한 한계를 극복하기 위해, 향후 연구에서는 강화학습 기법의 도입이 필요하다. 기존 모방학습으로 사전 학습된 정책을 기반으로 하되, 블록 적재의 높이, 작업 소요 시간, 궤적의 효율성 등을 평가하는 보상 함수를 세밀하게 설계하여 추가적인 학습을 진행할 수 있다. 충분히 최적화된 보상 함수를 기반으로 로봇이 환경과 상호작용하며 스스로 최적의 경로를 탐색하게 한다면, 단순히 인간의 데모를 흉내 내는 것을 넘어 전문가의 조작보다 더 효율적이고 뛰어난 행동 패턴과 강건성을 확보할 수 있을 것으로 기대된다. [9]

5.3.2 Sim-to-Real 도입

실제 환경에서 데이터를 수집하고 DAgger를 통해 개입하는 과정에서 소모되는 막대한 시간과 물리적 제약을 탈피하기 위해, 디지털 트윈 기반의 시뮬레이션 환경 구축 및 Sim-to-Real 기술의 도입이 필수적이다. Isaac Sim이나 MuJoCo와 같은 물리 엔진 시뮬레이터 내에서 로봇과 작업 환경을 동일하게 모델링하고, 시뮬레이션 상에서 수만 번의 에피소드를 병렬로 고속 수행하여 최적의 궤적을 찾고 정책을 학습시킬 수 있다. 향후 도메인 무작위화

기법을 적용하여 가상 공간에서 학습된 강건한 정책을 실제 로봇에 효과적으로 전이시킨다면, 데이터 수집 비용을 혁신적으로 절감함과 동시에 모델의 고도화를 달성할 수 있을 것이다.

[10]

제6장 추진 체계 및 수행 결과

6.1 구성원별 수행 결과

본 프로젝트를 위한 구성원별 역할 및 최종 수행 결과는 표 6.1와 같다.

이름	주요 수행 내용
윤주연	· 모방학습을 위한 Pick & Place 데이터셋 수집 · 모델 분석 및 하이퍼파라미터 테스트 · 서버를 통한 모델 학습 환경 구축
강태훈	· LeRobot SO-101 하드웨어 구성 · 데이터셋 수집 전략 수립 및 수정 · DAgger Correction을 위한 경로 수정 데이터셋 수집
송명근	· LeRobot SO-101 제어 환경 구축 · 다중 과제 해결을 위한 멀티태스크 환경 구축 · 모델 학습 및 수행 결과 분석

표 6.1: 구성원별 역할 및 수행 결과

6.2 개발 수행 일정

본 프로젝트는 6월부터 9월까지 총 4개월간 진행되었다. 상세한 추진 체계 및 월별·주차별 실제 수행 일정은 다음 표와 같다.

수행 항목	6월				7월				8월				9월			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
제어 모델 및 환경 구축 · SO-101 제어 모델 선정 · 프로젝트 개발 환경 구축																
데이터셋 수집 · Pick & Stack 시연 진행 · Hugging Face 포맷 업로드																
모델 학습 및 검증 · 데이터셋 증량 작업 · 데이터셋 기반 모델 생성 · 모델 성능 검증																
프로젝트 결과 정리 · 최종 결과 분석 · 향후 연구 방향 정리																

표 6.2: 추진 체계 및 실제 수행 일정

참고문헌

- [1] Hugging Face, “LeRobot: An Open-source Library for End-to-end Robot Learning,” GitHub Repository. <https://github.com/huggingface/lerobot>.
- [2] Hugging Face, “LeRobot SO-101 Hardware Stack Documentation,” <https://huggingface.co/docs/lerobot/so101>.
- [3] NVIDIA, “Get Started with Jetson Orin Nano Developer Kit,” <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>, (accessed Sep. 5, 2026).
- [4] Jetson AI Lab, “LeRobot on Jetson (Docker),” <https://www.jetson-ai-lab.com/archive/lerobot.html>.
- [5] LeRobot Community, “svla_so100_pickplace Dataset,” Hugging Face Datasets. https://huggingface.co/datasets/lerobot/svla_so100_pickplace.
- [6] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” arXiv preprint arXiv:2304.13705, 2023. <https://arxiv.org/abs/2304.13705>.
- [7] S. Chen, “Train an ACT Policy on SO-101,” Hugging Face Blog. <https://huggingface.co/blog/sherryxychen/train-act-on-so-101>.
- [8] HG-Dagger: Interactive Imitation Learning with Human Experts <https://www.alphaxiv.org/abs/1810.02890>
- [9] IEEE, “Related Research on Reinforcement Learning in Robotics,” <https://ieeexplore.ieee.org/document/10000161>.
- [10] NVIDIA Developer Community, “Bridging Virtual and Physical Robotics: An AI-Driven Educational Platform Using NVIDIA Omniverse Isaac Sim and Jetson Orin

Nano,” https://developer.nvidia.com/embedded/community/jetson-projects/sim_to_real.